

Binding Security of Implicitly-Rejecting KEMs and Application to BIKE and HQC

Juliane Krämer¹ , Patrick Struck²  and Maximiliane Weishäupl¹

¹ University of Regensburg, Regensburg, Germany

² University of Konstanz, Konstanz, Germany

Abstract. In this work, we continue the analysis of the binding properties of implicitly-rejecting key-encapsulation mechanisms (KEMs) obtained via the Fujisaki-Okamoto (FO) transform. These binding properties, in earlier literature known under the term robustness, thwart attacks that can arise when using KEMs in complex protocols. Recently, Cremers et al. (CCS’24) introduced a framework for binding notions, encompassing previously existing but also new ones. While implicitly-rejecting FO-KEMs have been analyzed with respect to multiple of these notions, there are still several gaps. We complete the picture by providing positive and negative results for the remaining notions. Further, we show how to apply our results to the code-based KEMs BIKE and HQC, which were round-4 candidates in NIST’s PQC standardization process. Through this, we close a second gap as our results complete the analysis of the binding notions for the NIST round-4 KEMs. Finally, we give a modified version of the FO transform that achieves all binding notions.

1 Introduction

Encryption is unequivocally the most fundamental concept of cryptography. Nowadays, encryption can be divided into symmetric encryption and asymmetric (public-key) encryption. For efficiency reasons, public-key encryption is typically used to encrypt only a symmetric key (of some fixed length), while the actual payload will be encrypted using that symmetric key. That is, public-key encryption (PKE) is essentially used to encrypt uniformly random messages that are used as symmetric keys. Key-encapsulation mechanisms (KEMs) are a primitive dedicated to this use-case: here, the encapsulation algorithm only takes a public key pk as input and outputs a uniform symmetric key k alongside a ciphertext c that encapsulates the symmetric key.

The standard security notion that KEMs should achieve is IND-CCA security, which is also the requirement of the NIST standardization process [NIS17]. The common method to obtain IND-CCA secure KEMs is to design IND-CPA secure public-key encryption schemes and then apply the Fujisaki-Okamoto (FO) transform [FO99] to it—a method that is used for virtually all KEMs in the NIST standardization process for post-quantum schemes [NIS17]. The FO transform comes in different variants. In this work, we are mainly concerned with the variant $\text{FO}^\perp$, which computes the shared key as $H(m, c)$ while ciphertexts are rejected by outputting $H(\sigma, c)$ instead, where σ is the implicit rejection value contained in the secret key. Another variant (also covered in this work) is $\text{FO}_m^\perp$,

E-mail: juliane.kraemer@ur.de (Juliane Krämer), patrick.struck@uni.kn (Patrick Struck), maximiliane.weishaepf@ur.de (Maximiliane Weishäupl)

*We thank Rune Fiedler for helpful discussions regarding the extension of our results to the LEAK^{+r} attack model. This work was funded by the Deutsche Forschungsgemeinschaft (DFG – German Research Foundation) – 505500359, by the Hector Foundation II, and by the German Federal Ministry of Research, Technology and Space (BMFTR) under the project Quant-ID (16KISQ111) and QUDIS (16KIS2091).

which works similar except that the shared key is computed as $H(m)$ as opposed to $H(m, c)$. The explicitly-rejecting variants $FO^\perp$ and $FO_m^\perp$, which are not in the scope of this work, are defined analogously to their implicitly-rejecting counterparts except that invalid ciphertext are rejected by outputting $\perp$ rather than $H(\sigma, c)$.

Using cryptographic primitives in larger protocols opens up room for misuse which is not prevented by the standard security notions. Several attacks show that this is a problem for digital signatures [JCCS19, Aye15] as well as authenticated encryption [DGRW18, ADG⁺22, LGR21]. Very recently, this was also shown to be a problem for KEMs: Cremers et al. [CDM24] showed that the authentication protocol presented in [BDK⁺18] is vulnerable to what is called a “re-encapsulation attack”.¹ Here, an adversary can decapsulate a ciphertext c to obtain a shared key k and then “re-encapsulate” that shared key under a different public key. This attack allows an adversary Eve to convince Alice that she shares a key with Eve although the key is actually shared between Alice and Bob—and unknown to Eve. While this does not compromise default security properties like confidentiality, it violates the so-called *implicit key agreement* that the protocol aims to achieve.

To deal with the general threat of misuse, more advanced security properties for core cryptographic primitives have emerged and enjoyed a lot of research in recent years. For authenticated encryption schemes, committing security prevents adversaries from finding ciphertexts that decrypt under more than one key, for which there is already a long line of research, e.g., [BH22, MLGR23, BH24, SW24, KSW24]. For digital signatures, BUFF security [CDF⁺21] prevents against various attacks that go beyond the unforgeability features of digital signatures and received a lot of attention lately [ADM⁺24, DFHS24, DFF24, DFH⁺24]. Both committing security and BUFF security are considered in the new standardization processes by NIST [NIS24, NIS22]. Very recently, Cremers et al. [CDM24]—motivated by the re-encapsulation attack—developed a framework for so-called “binding properties” of KEMs which prevent against various attacks. Note that this framework encompasses also the already existing robustness notions [GMP22], but renames them to match the new notation.

All notions are built following the pattern X-BIND-P-Q, where P and Q describe which elements (P) are binding which other elements (Q). An example (and one notion that we are considering in this work) is that P equals the shared key k of a KEM and Q equals the public key pk . The corresponding binding notion X-BIND-K-PK then formalizes that k binds pk or, simply speaking, an adversary cannot find ciphertexts that decapsulate to the same shared key k under distinct public keys. The other variable, X, describes the attack model and differs in how the keys are selected. Cremers et al. [CDM24] distinguish three cases: the malicious (MAL) setting where the adversary can generate the keys arbitrarily, the leaked (LEAK) setting where the keys are honestly generated and the adversary receives *all* keys, and the honest (HON) setting where the keys are honestly generated but the adversary only receives the public key along with access to decapsulation oracles for the secret keys. Thus, the three attack models exhibit the hierarchy MAL→LEAK→HON ordered from strong to weak. In total, Cremers et al. [CDM24] give 18 different security notions, which arise as the sensible combinations involving the shared key (K), the public key (PK), and the ciphertext (CT). These 18 notions can be expressed by the 6 general notions X-BIND-K-PK, X-BIND-K,CT-PK, X-BIND-K-CT, X-BIND-K,PK-CT, X-BIND-CT-PK, and X-BIND-CT-K for $X \in \{\text{MAL}, \text{LEAK}, \text{HON}\}$. Note that for X-BIND-K,CT-PK (and analogously for the other notions that include all three components K, PK, and CT), the notation is to be understood as $P = \{K, CT\}$ binding $Q = \{PK\}$.

There is a number of results regarding the security of implicitly-rejecting KEMs with respect to the binding notions. In the original work, Cremers et al. [CDM24] show

¹Note that this attack assumes the instantiation of the protocol with a KEM that does not fulfill the necessary binding notions—the instantiation with Kyber presented in [BDK⁺18] is not vulnerable.

that the ciphertext cannot bind any other value, i.e., the six notions X-BIND-CT-PK and X-BIND-CT-K with $X \in \{\text{MAL}, \text{LEAK}, \text{HON}\}$ are unachievable. The reason is that implicitly-rejecting KEMs always output some key $k \neq \perp$. We will generally exclude these six notions in this work which leaves us with 12 notions. Further, it is a direct implication of [CDM24, Theorem D.1] that schemes which use the FO-variant FO^χ do achieve X-BIND-K-CT (and hence also X-BIND-K,PK-CT) security for $X \in \{\text{MAL}, \text{LEAK}, \text{HON}\}$. Lastly, Grubbs et al. [GMP22, Theorem 6] prove that FO^χ -KEMs do fulfill HON-BIND-K,CT-PK security provided that a certain collision-freeness property holds for the underlying PKE scheme. With respect to FO_m^χ , Cremers et al. [CDM24] show that it achieves LEAK-BIND-K,PK-CT security (hence also HON-BIND-K,PK-CT security) while it is neither X-BIND-K-PK nor X-BIND-K-CT secure (for any $X \in \{\text{HON}, \text{LEAK}, \text{MAL}\}$).² This leaves several notions open for FO^χ and FO_m^χ . Figure 1 provides an overview of the notions considered in this work, as well as the existing and our new results for FO^χ and FO_m^χ .

Next to these general results, the binding notions have been (partially) analyzed for several of the KEMs included in NIST’s standardization process: Grubbs et al. [GMP22] analyzed the security of CLASSIC-McELIECE [ABC⁺22], HQC [AAB⁺20] (round-3 version), SABER [DKR⁺20], FRODOKEM [NAB⁺20], and KYBER [BDK⁺18] with respect to some of the binding notions. The analysis of those four schemes and ML-KEM [NIS23] was extended by Cremers et al. [CDM24] and more results for ML-KEM were given by Schmieg [Sch24]. The remaining two round-4 KEMs BIKE [ABB⁺22] and the current version of HQC [AAB⁺24]³ (which was very recently selected for standardization by NIST), however, are yet to be analyzed with respect to *any* of the binding properties.⁴

1.1 Contribution

In this work, we close both of the aforementioned gaps. We complete the analysis of the binding notions for implicitly-rejecting KEMs, i.e., those resulting from either FO^χ or FO_m^χ . An overview of the results is provided in Table 1 and further illustration is given in Figure 1. Further, we complete the binding analysis of the NIST round-4 KEMs with the results being depicted in Table 2. Finally, we provide different variants of the FO transform (cf. Table 3) that achieve all notions with respect to the different attack models (HON/LEAK/MAL)—two are established variants, while the strongest one is newly developed by us.

Regarding FO^χ -KEMs, we show the following. Firstly, for FO^χ -KEMs, we prove equivalence between HON-BIND-K-PK and the (weaker) notion HON-BIND-K,CT-PK.⁵ This allows us to then leverage [GMP22, Theorem 6], which provides requirements for FO^χ -KEMs to achieve HON-BIND-K,CT-PK. The core requirement is that the underlying PKE scheme needs to satisfy a weak form of collision-freeness, called SCFR-CPA. Secondly, we formalize a property of the underlying PKE scheme, called restricted non-rigidity, such that the KEM (resulting from applying the FO^χ transform) is vulnerable with respect to any of the four notions LEAK-BIND-K-PK, LEAK-BIND-K,CT-PK, MAL-BIND-K-PK, and MAL-BIND-K,CT-PK. For the latter two we also give another attack that does not impose any requirement on the underlying encryption. Thirdly, we show that FO^χ -KEMs do achieve MAL-BIND-K-CT and MAL-BIND-K,PK-CT security—again not relying on any requirements for the underlying encryption. An overview of these results can be found in Table 1.

We further show that these results can be applied to the code-based key-encapsulation

²We exclude the results that are based on a conjecture.

³The relevant change for our work is that round-4 HQC uses implicit rejection, whereas round-3 HQC uses explicit rejection.

⁴Also a few notions for CLASSIC-McELIECE and ML-KEM remain without analysis.

⁵In fact, we also prove this for the corresponding LEAK notions.

Table 1: Overview of our results for the binding notions of implicitly-rejecting KEMs obtained via either of the two FO transforms. Positive results are marked with $\checkmark$ while negative ones are marked with $\times$; an asterisk, i.e., $\checkmark^*/\times^*$, indicates that a result (either positive or negative) relies on some mild assumption. Results from prior works, i.e., Theorem 1, Theorem 2, Theorem 3, and Theorem 4, are written in parentheses.

General Notion	Notion	$\text{FO}^\mathcal{L}$		$\text{FO}_m^\mathcal{L}$
X-BIND-K-PK	MAL-BIND-K-PK	$\times$	Thm. 5	$\times$ (Thm. 4)
	LEAK-BIND-K-PK	$\times^*$	Thm. 5	$\times$ (Thm. 4)
	HON-BIND-K-PK	$\checkmark^*$	Thm. 6	$\times$ (Thm. 4)
X-BIND-K,CT-PK	MAL-BIND-K,CT-PK	$\times$	Thm. 8	$\times$ Thm. 8
	LEAK-BIND-K,CT-PK	$\times^*$	Thm. 5	$\checkmark^*$ Thm. 16
	HON-BIND-K,CT-PK	$\checkmark^*$	(Thm. 1)	$\checkmark^*$ Thm. 16
X-BIND-K-CT	MAL-BIND-K-CT	$\checkmark$	Thm. 7	$\times$ (Thm. 4)
	LEAK-BIND-K-CT	$\checkmark$	(Thm. 2)	$\times$ (Thm. 4)
	HON-BIND-K-CT	$\checkmark$	(Thm. 2)	$\times$ (Thm. 4)
X-BIND-K,PK-CT	MAL-BIND-K,PK-CT	$\checkmark$	Thm. 7	$\checkmark$ Thm. 18
	LEAK-BIND-K,PK-CT	$\checkmark$	(Thm. 2)	$\checkmark$ (Thm. 4)
	HON-BIND-K,PK-CT	$\checkmark$	(Thm. 2)	$\checkmark$ (Thm. 4)

mechanism BIKE [ABB⁺22], as it applies the $\text{FO}^\mathcal{L}$ transform. For the code-based scheme HQC [AAB⁺22] some of the binding notions require a dedicated analysis due to HQC using a modified variant of the $\text{FO}^\mathcal{L}$ transform. Resulting from this, we observe that HQC achieves only 2 out of 12 notions, in particular, none of the LEAK or MAL notions. Next to this, we also describe a slight modification to HQC, resulting in a scheme we call HQC*, which provides more binding security than HQC. More precisely, we observe that the general $\text{FO}^\mathcal{L}$ results can be applied not only for BIKE but also for HQC*, i.e., it follows from the existing and our new results that both schemes fulfill 8 out of 12 notions: all HON notions plus X-BIND-K-CT and X-BIND-K,PK-CT for $X \in \{\text{MAL}, \text{LEAK}\}$. An overview of these results can be found in Table 2.

Regarding $\text{FO}_m^\mathcal{L}$ -KEMs, we have the following results. We show that $\text{FO}_m^\mathcal{L}$ -KEMs do achieve MAL-BIND-K,PK-CT security while they do not achieve MAL-BIND-K,CT-PK security. However, we have a positive result with respect to LEAK-BIND-K,CT-PK (and hence HON-BIND-K,CT-PK) based on an additional requirement of the underlying encryption scheme which is achieved by the encryption schemes underlying BIKE and HQC. Furthermore, we prove that HON-BIND-K-PK and HON-BIND-K,CT-PK (and their corresponding LEAK-variants LEAK-BIND-K-PK and LEAK-BIND-K,CT-PK) are not equivalent which is in contrast to $\text{FO}^\mathcal{L}$ -KEMs, where we show this to be the case. These results are included in Table 1.

Given that neither $\text{FO}^\mathcal{L}$ nor $\text{FO}_m^\mathcal{L}$ achieve all binding notions, we describe three transforms FO_H , FO_L , and FO_M , that achieve all binding notions wrt different attack models: (1) FO_H is simply the $\text{FO}^\mathcal{L}$ transform which, using some mild assumption on the underlying PKE scheme, achieves all HON notions; (2) FO_L is a variant of the $\text{FO}^\mathcal{L}$ transform close to the one used by ML-KEM and can be seen to achieve all LEAK notions; (3) FO_M is a slight modification of FO_L (the only change is that the public key is also hashed in the rejecting case) which we show to achieve all MAL notions—by the hierarchy also all LEAK and HON notions. Table 3 provides an overview of these transforms.

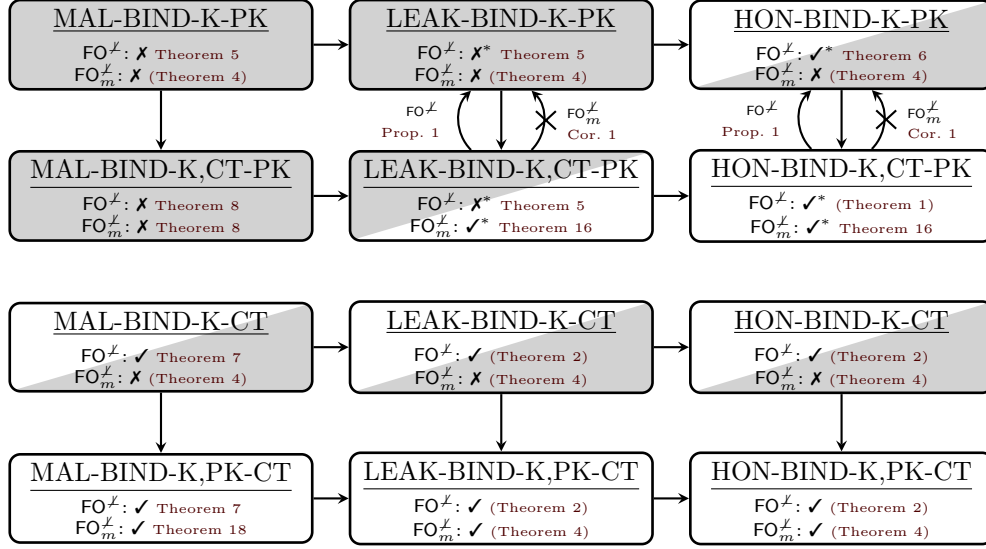


Figure 1: Overview of the 12 binding notions for key-encapsulation mechanisms with their implications. For each notion, we state whether it is achieved ($\checkmark$) or not achieved ($\times$) by FO^X or FO_m^X . If the result relies on an extra condition, it is indicated by an asterisk ($\checkmark^*/\times^*$). Results from prior works, i.e., Theorem 1, Theorem 2, Theorem 3, and Theorem 4, are written in parentheses; further, boxes with positive results are left white, while ones with negative results are marked in gray. Boxes with mixed results have both colors; the top left color indicates the result for FO^X while the bottom right color indicates the result for FO_m^X . Arrows indicate implications while crossed arrow denote separations.

1.2 Related Work

The binding properties generalize the concept of robust encryption, which can be traced back to [ABN10] and ensures that it is hard for an adversary to produce a ciphertext that is valid under more than one key. Robust encryption plays a key role for achieving anonymity of public-key encryption. Anonymity was introduced in [BBDP01] and captures the idea that an adversary, who obtains a ciphertext, cannot distinguish which key was used to generate said ciphertext.

Grubbs et al. [GMP22] and Xagawa [Xag22] studied the anonymity and robustness of post-quantum secure KEMs: the former analyzed robustness for three NIST finalists (CLASSIC-McELIECE [ABC⁺20], SABER [DKR⁺20], and KYBER [SAB⁺20]) and one alternate candidate (FRODOKEM [NAB⁺20]); the latter analyzed the anonymity for all round-3 KEMs.

Cremers et al. [CDM24] provide a framework covering various binding notions for KEMs which subsumes existing robustness properties. Similar properties were introduced in [BCD⁺24] and [AHK⁺23] which are covered by the general framework from [CDM24]. Besides the introduction of the framework, Cremers et al. [CDM24] also cover a (partial) analysis of KYBER, FRODOKEM, CLASSIC-McELIECE, and ML-KEM. For the latter, ML-KEM, Schmieg [Sch24] analyzed some of the strongest binding notions. Fiedler and Günther [FG25] introduce a fourth attack model for the binding properties analyzing the Signal’s PQXDH handshake.

Table 2: Binding security of the round-4 KEMs BIKE, HQC, HQC*, and CLASSIC-McELIECE, as well as ML-KEM. Note that HQC* is a slightly modified version of HQC, which we introduce in this paper.

Notion	BIKE	HQC / HQC*	CL-McELIECE	ML-KEM
MAL-BIND-K-PK	✗	✗ / ✗	✗	✗
LEAK-BIND-K-PK	✗	✗ / ✗	✗	✓
HON-BIND-K-PK	✓	✓ / ✓	✗	✓
MAL-BIND-K,CT-PK	✗	✗ / ✗	✗	✗
LEAK-BIND-K,CT-PK	✗	✗ / ✗	✗	✓
HON-BIND-K,CT-PK	✓	✓ / ✓	✗	✓
MAL-BIND-K-CT	✓	✗ / ✓	✓	✗
LEAK-BIND-K-CT	✓	✗ / ✓	✓	✓
HON-BIND-K-CT	✓	✗ / ✓	✓	✓
MAL-BIND-K,PK-CT	✓	✗ / ✓	✓	✓
LEAK-BIND-K,PK-CT	✓	✗ / ✓	✓	✓
HON-BIND-K,PK-CT	✓	✗ / ✓	✓	✓

Table 3: Overview on variants of the FO transform and what notions they achieve. A ✓ means that *all* notion are achieved in the respective attack model, while a ✗ indicates that *at least one* notion in the respective attack model is not achieved. An asterisk denotes that the result relies on a mild assumption. Here, h_{pk} denotes the hash of the public key.

Transform	Computation of the Shared Key		Attack Model		
	accept	reject	HON	LEAK	MAL
FO _H	$H(m, c)$	$H(\sigma, c)$	✓*	✗	✗
FO _L	$H(m, h_{pk})$	$H(\sigma, c)$	✓	✓	✗
FO _M	$H(m, h_{pk})$	$H(\sigma, h_{pk}, c)$	✓	✓	✓

2 Background

This section provides the necessary background for this work. [Section 2.1](#) describes the notation used throughout. The definitions for public-key encryption (PKE) schemes and key-encapsulation mechanisms (KEM) are given in [Section 2.2](#) while the Fujisaki-Okamoto transform to turn a PKE into a KEM is described in [Section 2.3](#). The various binding notions covered in this work appear in [Section 2.4](#).

2.1 Notation

For a set $\mathcal{S}$, $s \leftarrow \mathcal{S}$ denotes that a uniform random element from $\mathcal{S}$ is chosen and assigned to s . For a distribution D , we write $x \leftarrow D$ to indicate that x is drawn according to D . For a randomized algorithm ALG , we write $y \leftarrow \text{ALG}(x)$ to denote that y is the output of ALG on input x . Often we will be explicit about the random coins of ALG in which case we write $y \leftarrow \text{ALG}(x; r)$ (this means that $y \leftarrow \text{ALG}(x)$ picks uniformly random coins r and computes $y \leftarrow \text{ALG}(x; r)$). Throughout this work, we implicitly consider the security parameter λ and will hence not write it as an explicit input for algorithms. Whenever we write adversary $\mathcal{A}$, it means an algorithm that runs in polynomial-time in the security

parameter and security wrt to any notion means that an adversary succeeds at most with negligible probability in the security parameter. For some security notions (those of the form MAL-BIND-P-Q), the security game is a two-stage game for which we assume that the adversaries share an implicit state.

2.2 Public-Key Encryption and Key-Encapsulation Mechanisms

Below we define public-key encryption and key-encapsulation mechanisms.

Definition 1. A public-key encryption scheme PKE consists of three algorithms KEYGEN, ENC, and DEC, where

KEYGEN() is the key generation algorithm that outputs a key pair consisting of a public key pk and a secret key sk .

ENC(pk, m) is the encryption algorithm which takes as input a public key pk and a message m , and outputs a ciphertext c .

DEC(sk, c) is the decryption algorithm which takes as input a secret key sk and a ciphertext c , and outputs a message m (or a special failure symbol $\perp$).

Definition 2. A key-encapsulation mechanism (KEM) KEM is a triple of three algorithms (KEYGEN, ENCAPS, DECAPS) where

KEYGEN() is the key generation algorithm that outputs a key pair consisting of a public key pk and a secret key sk .

ENCAPS(pk) is the encapsulation algorithm which takes as input a public key pk , and outputs a ciphertext c and an encapsulated key k .

DECAPS(sk, c) is the decapsulation algorithm which takes as input a secret key sk and a ciphertext c , and outputs a key k (or a special failure symbol $\perp$).

We write $\mathcal{M}$ for the message space of a public-key encryption scheme, which—for FO-KEMs that we consider in this work—coincides with the randomness space of a key-encapsulation mechanism. As mentioned above, we will typically be explicit about the random coins. This entails that we write $c \leftarrow \text{ENC}(pk, m; r)$ to indicate that c is deterministically computed using r as random coins. Likewise, we write $(c, k) \leftarrow \text{ENCAPS}(pk; r)$ to indicate that (c, k) is deterministically computed using random coins r .

During this work, we will implicitly assume that the PKE schemes achieve IND-CPA security (and hence the corresponding FO-transformed scheme is IND-CCA secure) to guarantee that a random ciphertext will be invalid with overwhelming probability—this property is also exploited in the IND-CCA security proof of the FO transform. We assume that the public-key space is superpolynomial and that the PKE schemes and KEMs are correct with overwhelming probability, i.e., for honestly generated ciphertexts, DEC and DECAPS return the correct message m and key k with overwhelming probability, respectively. We further assume, for both PKE schemes and KEMs, that the public key pk can be derived from the corresponding secret key sk —for honestly generated key-pairs. This is in line with the requirement by NIST. We note that the theoretical analysis then allows the attacker to maliciously choose the actual public key different from the public key contained in the secret key; in practice, however, (part of) the public key is often computed from a shorter seed and the secret key will only contain this seed but not the whole public key.

2.3 The Fujisaki-Okamoto Transform

The Fujisaki-Okamoto transform [FO99] turns a weakly secure public-key encryption scheme into a strongly secure public-key encryption scheme. A variant which transforms a PKE scheme into a KEM was first given by Dent [Den03]. In this work, we consider the modular versions of it given in [HHK17]. Here, the transform is composed of two transforms: T_g and U . The transform T_g derandomizes a public-key encryption scheme by deriving the random coins used for encryption from the message. To validate ciphertexts, the decryption additionally checks if re-encrypting a decrypted ciphertext results in the same ciphertext. The transform U transforms a derandomized public-key encryption scheme into a KEM. Here, a random message is chosen and encrypted using the derandomized PKE scheme resulting in a ciphertext. The shared key is obtained by hashing the random message (and this ciphertext). For an FO-KEM, the randomness is a message for the underlying PKE scheme, hence we will typically write $\text{KEM.ENCAPS}(pk; m)$ instead of $\text{KEM.ENCAPS}(pk; r)$. The transform U comes in four different variants $U^\perp$, $U_m^\perp$, $U^\perp$, and $U_m^\perp$, where the first two are using implicit rejection while the latter two use explicit rejection. Composing the T_g transform with the four variants of the U transform yields the four FO variants $\text{FO}^\perp$, $\text{FO}_m^\perp$, $\text{FO}^\perp$, and $\text{FO}_m^\perp$.

The $\text{FO}^\perp$ transform is currently the most commonly used variant, being deployed by all of the round-4 NIST KEMs—in particular by BIKE and HQC⁶, which we will consider in Section 4. This variant (decomposed into the two underlying transforms T_g and $U^\perp$) is shown in Figure 2 along with the $\text{FO}_m^\perp$ transform (decomposed into the two underlying transforms T_g and $U_m^\perp$).

However, during the course of the NIST standardization process, also other FO-variants were used in the submitted KEMs. For instance, LEDACRYPT [BBC⁺19] and NTS-KEM [ACP⁺19] both relied on $\text{FO}_m^\perp$, while HQC in round-3 used $\text{FO}^\perp$. In view of the trend towards implicit rejection, we mainly focus on $\text{FO}_m^\perp$ and $\text{FO}^\perp$ in this paper.

The security of the FO transform relies on H_T and H_U to be random oracles. Our results rely on the same assumption and throughout this work, we model H_T and H_U as random oracles without explicitly stating it each time.

2.4 Binding Properties of Key-Encapsulation Mechanisms

Cremers et al. [CDM24] developed a framework for binding properties of KEMs. Binding notions in this framework are of the form X-BIND-P-Q. Here, P and Q describe which elements (P) are binding which other elements (Q) and $X \in \{\text{MAL}, \text{LEAK}, \text{HON}\}$ determines the adversary model, more precisely, how the keys are selected.⁷ The generic binding security game X-BIND-P-Q is shown in Figure 3.⁸ For implicitly-rejecting KEMs, there are a total of twelve binding notions: X-BIND-K-PK and X-BIND-K,CT-PK as well as X-BIND-K-CT and X-BIND-K,PK-CT for $X \in \{\text{HON}, \text{LEAK}, \text{MAL}\}$ that can be derived from the generic games. For convenience, we also provide the explicit games in Figure 8 and Figure 9 in Appendix A. The relations between the notions are described in Figure 1. Below we define security wrt the various binding notions.

Definition 3. Let KEM be a key-encapsulation mechanism. Further consider $X \in \{\text{HON}, \text{LEAK}, \text{MAL}\}$, $P \in \{\{K\}, \{K, \text{CT}\}\}$, and the game X-BIND-P-PK defined in Figure 3. We say that KEM achieves X-BIND-P-PK if for any adversary $\mathcal{A}$, its probability in winning X-BIND-P-PK is negligible.

⁶Note, however, that HQC deploys a modified $\text{FO}^\perp$ version.

⁷A fourth case was introduced by Fiedler and Günther [FG25]. In this case, dubbed LEAK^{+r} , the adversary is not provided with the key-pairs but the randomness used to generate those; this puts it between LEAK and MAL.

⁸SCFR-CCA and SROB-CCA from [GMP22] correspond to HON-BIND-K,CT-PK and HON-BIND-CT-PK, respectively, in the framework from [CDM24].

$\text{PKE}^{\mathcal{S}}.\text{KEYGEN}()$	$\text{PKE}^{\mathcal{S}}.\text{ENC}(pk, m)$	$\text{PKE}^{\mathcal{S}}.\text{DEC}(sk, c)$
$(pk, sk) \leftarrow \text{KEYGEN}()$	$c \leftarrow \text{ENC}(pk, m; \text{H}_{\top}(m))$	$m \leftarrow \text{DEC}(sk, c)$
return (pk, sk)	return c	$\bar{c} \leftarrow \text{ENC}(pk, m; \text{H}_{\top}(m))$
		if $\bar{c} \neq c$
		return $\perp$
		return m
$\text{KEYGEN}^{\mathcal{L}}()$	$\text{ENCAPS}(pk)$	$\text{DECAPS}(sk^{\mathcal{L}}, c)$
$(pk, sk) \leftarrow \text{KEYGEN}()$	$m \leftarrow \mathcal{M}$	$(sk, \sigma) \leftarrow sk^{\mathcal{L}}$
$\sigma \leftarrow \mathcal{M}$	$c \leftarrow \text{PKE}^{\mathcal{S}}.\text{ENC}(pk, m)$	$m \leftarrow \text{PKE}^{\mathcal{S}}.\text{DEC}(sk, c)$
$sk^{\mathcal{L}} \leftarrow (sk, \sigma)$	$k \leftarrow \text{H}_{\text{U}}(m, c) \quad // \text{U}^{\mathcal{L}}$	if $m \neq \perp$
return $(pk, sk^{\mathcal{L}})$	$k \leftarrow \text{H}_{\text{U}}(m) \quad // \text{U}_m^{\mathcal{L}}$	return $\text{H}_{\text{U}}(m, c) \quad // \text{U}^{\mathcal{L}}$
	return (c, k)	return $\text{H}_{\text{U}}(m) \quad // \text{U}_m^{\mathcal{L}}$
		return $\text{H}_{\text{U}}(\sigma, c)$

Figure 2: Top: The derandomized PKE scheme $\text{PKE}^{\mathcal{S}} = \text{T}_{\mathcal{S}}[\text{PKE}, \text{H}_{\top}]$ constructed from a PKE scheme PKE and a random oracle $\text{H}_{\top}$. **Bottom:** The implicitly-rejecting KEMs $\text{U}^{\mathcal{L}}[\text{PKE}^{\mathcal{S}}, \text{H}_{\text{U}}]$ and $\text{U}_m^{\mathcal{L}}[\text{PKE}^{\mathcal{S}}, \text{H}_{\text{U}}]$.

Remark 1. In our formalization of the MAL notions, the adversary outputs either the secret key *or* the public key of a key pair, but never both at the same time. Since we assume that the secret key contains the public key, the re-encryption step during decapsulation is still possible. More precisely, for the different variants in the MAL games (see Figure 3), the outputs are (I) $sk, \overline{sk}$; (II) $pk, \overline{sk}$; and (III) $pk, \overline{pk}$. This is equivalent to the formalization by Cremers et al. [CDM24] where the adversary has to output two key pairs (pk, sk) and $(\overline{pk}, \overline{sk})$ and the public key is an explicit input to the decapsulation algorithm.⁹

For some of our results, we need certain requirements for the underlying PKE scheme. More precisely, we require a form of collision-freeness which we establish in three different variants: SCFR-CPA, SCFR-CCA, and SCFR-LEAK as defined below. Note that SCFR-CPA and SCFR-CCA were introduced in [GMP22], while SCFR-LEAK is a new notion introduced in this work.¹⁰

Definition 4. Consider the games SCFR-CPA, SCFR-CCA, and SCFR-LEAK as defined in Figure 4. We call a public-key encryption scheme PKE SCFR-CPA, SCFR-CCA, and SCFR-LEAK secure, if for any adversary its probability of winning the corresponding game is negligible.

In the following we recall some existing results. The first one by Grubbs et al. [GMP22] establishes HON-BIND-K,CT-PK security for KEMs via the $\text{FO}^{\mathcal{L}}$ transform assuming SCFR-CPA security of the underlying (derandomized) PKE scheme. The other two are general results about implicitly-rejecting FO transforms given by Cremers et al. [CDM24].

Theorem 1 (Adapted from [GMP22, Theorem 6]). *Let PKE be a public-key encryption scheme that has negligible decryption errors and $\text{H}_{\top}$ and H_{U} be random oracles. If $\text{PKE}^{\mathcal{S}} = \text{T}_{\mathcal{S}}[\text{PKE}, \text{H}_{\top}]$ is SCFR-CPA secure, then $\text{KEM}^{\mathcal{L}} = \text{FO}^{\mathcal{L}}[\text{PKE}, \text{H}_{\top}, \text{H}_{\text{U}}]$ is secure wrt the notion HON-BIND-K,CT-PK.*

⁹Note that an earlier version of [CDM24] did not consider the public key as an explicit input to the decapsulation algorithm which led to MAL attacks as shown by Schmieg [Sch24].

¹⁰Note that the PKE schemes underlying BKE and HQC fulfill the new notion SCFR-LEAK (see Proposition 3).

Game X-BIND-P-Q	Game MAL-BIND-P-Q
$(pk, sk) \leftarrow \text{KEYGEN}()$	$g \leftarrow \mathcal{A}()$
$(\overline{pk}, \overline{sk}) \leftarrow \text{KEYGEN}()$	if $g = 1$: // Scenario (I): DECAPS-DECAPS
if $PK \in P$:	$(sk, \overline{sk}, c, \bar{c}) \leftarrow \mathcal{A}()$
$(\overline{pk}, \overline{sk}) \leftarrow (pk, sk)$	$k \leftarrow \text{DECAPS}(sk, c)$
if $PK \notin P \cup Q$:	$\bar{k} \leftarrow \text{DECAPS}(\overline{sk}, \bar{c})$
$b \in \{0, 1\} \leftarrow \mathcal{A}()$	if $g = 2$: // Scenario (II): ENCAPS-DECAPS
if $b = 0$: $(\overline{pk}, \overline{sk}) \leftarrow (pk, sk)$	$(pk, \overline{sk}, r, \bar{c}) \leftarrow \mathcal{A}()$
if $X = \text{HON}$:	$(k, c) \leftarrow \text{ENCAPS}(pk; r)$
$(c, \bar{c}) \leftarrow \mathcal{A}^{\text{Dec}, \overline{\text{Dec}}}(pk, \overline{pk})$	$\bar{k} \leftarrow \text{DECAPS}(\overline{sk}, \bar{c})$
if $X = \text{LEAK}$:	if $g \notin \{1, 2\}$: // Scenario (III): ENCAPS-ENCAPS
$(c, \bar{c}) \leftarrow \mathcal{A}(pk, sk, \overline{pk}, \overline{sk})$	$(pk, \overline{pk}, r, \bar{r}) \leftarrow \mathcal{A}()$
$k \leftarrow \text{DECAPS}(sk, c)$	$(k, c) \leftarrow \text{ENCAPS}(pk; r)$
$\bar{k} \leftarrow \text{DECAPS}(\overline{sk}, \bar{c})$	$(\bar{k}, \bar{c}) \leftarrow \text{ENCAPS}(\overline{pk}; \bar{r})$
if $k = \perp \vee \bar{k} = \perp$:	if $k = \perp \vee \bar{k} = \perp$:
return 0	return 0
$v_p \leftarrow (\forall p \in P : x_p = \bar{x}_p)$	$v_p \leftarrow (\forall p \in P : x_p = \bar{x}_p)$
$v_q \leftarrow (\exists q \in Q : x_q \neq \bar{x}_q)$	$v_q \leftarrow (\exists q \in Q : x_q \neq \bar{x}_q)$
return $v_p \wedge v_q$	return $v_p \wedge v_q$

Figure 3: Generic security games for the notions X-BIND-P-Q and MAL-BIND-P-Q for $P = \{K, \text{CT}, (K, \text{CT}), (K, \text{PK})\}$, $Q = \{\text{PK}, K, \text{CT}\}$, and $X \in \{\text{HON}, \text{LEAK}\}$. Note that for $p \in P$ (and $q \in Q$), we denote the corresponding instances by $x_p, \bar{x}_p$ (and $x_q, \bar{x}_q$, respectively). For example, if $p = \text{CT}$, we have $x_p = c$ and $\bar{x}_p = \bar{c}$. For variants (I) and (II) of the MAL notions, it is understood that any check of the form $pk \bowtie \overline{pk}$, for $\bowtie \in \{=, \neq\}$ is performed on the public keys contained in the respective secret keys sk and $\overline{sk}$. We refer to Remark 1 for more details. The decapsulation oracles Dec and $\overline{\text{Dec}}$ in case of $X = \text{HON}$ are omitted and work in the obvious way (without any restrictions) using sk and $\overline{sk}$, respectively.

Theorem 2 (Adapted from [CDM24, Theorem D.1]). *Let PKE be a public-key encryption scheme and H_T and H_U be random oracles. Then $\text{KEM}^\perp = \text{FO}^\perp[\text{PKE}, H_T, H_U]$ is LEAK-BIND-K-CT secure.*

Theorem 3 ([CDM24, Theorem 4.11]). *An implicitly-rejecting key-encapsulation mechanism KEM cannot be HON-BIND-CT-PK or HON-BIND-CT-K secure.*

Theorem 4 (Adapted from [CDM24, Section B.4]). *Let PKE be a public-key encryption scheme and H_T and H_U be random oracles. Then the key-encapsulation mechanism $\text{KEM}_m^\perp = \text{FO}_m^\perp[\text{PKE}, H_T, H_U]$ is LEAK-BIND-K,PK-CT secure, but insecure with respect to HON-BIND-K-CT and HON-BIND-K-PK.*

Remark 2. Note that [CDM24, Section B.4] also contains a positive result for the LEAK-BIND-K,CT-PK security of $\text{FO}_m^\perp$ -KEMs. However, a part of the proof is based on a conjecture and left open for future work—this is why we do not include the result in Theorem 4. Further details on this are provided in Section B.1.

Game SCFR-LEAK	Game SCFR-CCA	Oracle Dec(c)
$(pk, sk) \leftarrow \$ \text{KEYGEN}()$	$(pk, sk) \leftarrow \$ \text{KEYGEN}()$	$k \leftarrow \text{DECAPS}(sk, c)$
$(\overline{pk}, \overline{sk}) \leftarrow \$ \text{KEYGEN}()$	$(\overline{pk}, \overline{sk}) \leftarrow \$ \text{KEYGEN}()$	return k
$c \leftarrow \mathcal{A}(pk, \overline{pk}, sk, \overline{sk})$	$c \leftarrow \mathcal{A}^{\text{Dec}, \overline{\text{Dec}}}(pk, \overline{pk})$	Oracle $\overline{\text{Dec}}(c)$
$m \leftarrow \text{DEC}(sk, c)$	$m \leftarrow \text{DEC}(sk, c)$	$k \leftarrow \text{DECAPS}(\overline{sk}, c)$
$\overline{m} \leftarrow \text{DEC}(\overline{sk}, c)$	$\overline{m} \leftarrow \text{DEC}(\overline{sk}, c)$	return k
return $m = \overline{m} \neq \perp$	return $m = \overline{m} \neq \perp$	

Figure 4: Security games SCFR-LEAK and SCFR-CCA for PKEs. By removing the decapsulation oracles in game SCFR-CCA, we obtain the game SCFR-CPA as defined in [GMP22] and required for Theorem 1.

3 General Analysis of Implicitly-Rejecting FO

We give general results about the binding security of implicitly-rejecting FO-KEMs, i.e., $\text{FO}^\perp$ and $\text{FO}_m^\perp$. These results complete the binding analysis of the two implicitly-rejecting FO variants. Section 3.1 and Section 3.2 cover the notions of the form X-BIND-K,CT-PK and X-BIND-K-PK for $\text{FO}^\perp$. Section 3.3 completes the picture by showing security with respect to the notions MAL-BIND-K-CT and MAL-BIND-K,PK-CT for $\text{FO}^\perp$. Section 3.4 presents several MAL-BIND-P-Q attacks for both $\text{FO}^\perp$ and $\text{FO}_m^\perp$. The results for $\text{FO}_m^\perp$ are deferred to Appendix B. More precisely, Appendix B.1 covers the notions of the form X-BIND-K,CT-PK and Appendix B.2 the ones of the form X-BIND-K,PK-CT.

3.1 LEAK-BIND-K,CT-PK Attack for $\text{FO}^\perp$

Below we define a property of public-key encryption schemes needed for our results. It bears similarities with the rigidity property [BP18], where it is checked whether decrypting a ciphertext and then re-encrypting the resulting message will return the same ciphertext. However, in the definition below, we consider the negated version (non-rigidity) and add a further restriction: we consider only ciphertexts obtained from an honest encryption with a public key that is *different* from the one used for the actual non-rigidity check. This is why we call the property *restricted non-rigidity*.

Rigidity was initially introduced for the T_g transform underlying the FO transform. This transformation derandomizes a PKE scheme by deriving the random coins as the hash of the message. Looking ahead, we will need this property not only for the T_g transform but also for a modified version used by HQC, which we will refer to as T_s .

Definition 5. Let PKE be a public-key encryption scheme. We say that PKE fulfills *restricted non-rigidity* (or PKE is *restricted non-rigid*) if for two honestly generated key pairs $(pk, sk), (\overline{pk}, \overline{sk})$, a randomly chosen message m , and $c \leftarrow \$ \text{PKE.ENC}(pk, m)$, we have

$$\text{PKE.ENC}(\overline{pk}, \text{PKE.DEC}(\overline{sk}, c)) \neq c. \quad (1)$$

with overwhelming probability. Note that the probability is taken over the random coins used to generate the key pair, to pick the message, and for encryption.

Remark 3. For T_g the re-encrypt transform as defined in Figure 2, the property from Eq. (1) corresponds to the re-encrypt check failing. An honestly generated ciphertext using (pk, sk) will be invalid under a different key pair $(\overline{pk}, \overline{sk})$ with overwhelming probability.

The theorem below shows that applying $\text{FO}^\perp$ to a PKE scheme that is restricted non-rigid results in a KEM that is *not* LEAK-BIND-K,CT-PK. The attack is conceptually very simple: the adversary takes the implicit-rejection-value σ from one of the key pairs and

honestly encapsulates it under the other key pair, yielding a ciphertext c . By correctness c gets decapsulated to $H_U(\sigma, c)$ (under the key pair which was used to honestly generate c). By non-rigidity of the underlying PKE scheme, c gets rejected with overwhelming probability under the other key pair, which, however, will result in the same output $H_U(\sigma, c)$.

Theorem 5. *Let PKE be a public-key encryption scheme and $\text{KEM}^\mathcal{L}$ the key-encapsulation mechanism obtained from applying $\text{FO}^\mathcal{L}$ to PKE. If PKE is restricted non-rigid, then $\text{KEM}^\mathcal{L}$ is neither X-BIND-K,CT-PK nor X-BIND-K-PK secure for $X \in \{\text{MAL}, \text{LEAK}\}$.*

Proof. We construct a LEAK-BIND-K,CT-PK adversary $\mathcal{A}$ against $\text{KEM}^\mathcal{L}$ as follows. As input, $\mathcal{A}$ obtains two honestly generated key pairs $(pk, sk^\mathcal{L} = (sk, \sigma)), (\overline{pk}, \overline{sk}^\mathcal{L} = (\overline{sk}, \overline{\sigma})) \leftarrow^* \text{KEM}^\mathcal{L}.\text{KEYGEN}()$ and has to output c such that

$$\text{KEM}^\mathcal{L}.\text{DECAPS}(sk^\mathcal{L}, c) = k = \overline{k} = \text{KEM}^\mathcal{L}.\text{DECAPS}(\overline{sk}^\mathcal{L}, c).$$

Note that we have $pk \neq \overline{pk}$ with overwhelming probability. As a first step, $\mathcal{A}$ sets $m = \overline{\sigma}$ where $\overline{\sigma}$ is the implicit rejection value from $\overline{sk}^\mathcal{L}$ and computes an honest encapsulation of pk , using not a random message but m instead: $(c, k) \leftarrow \text{KEM}^\mathcal{L}.\text{ENCAPS}(pk; m)$. Then, $\mathcal{A}$ outputs c .

Below we argue that $\mathcal{A}$ wins the game LEAK-BIND-K,CT-PK with overwhelming probability. Note first that $k = H_U(m, \cdot)$ which follows by definition of $\text{KEM}^\mathcal{L}.\text{ENCAPS}$. Note further that $\text{KEM}^\mathcal{L}.\text{DECAPS}(sk^\mathcal{L}, c) = k$ holds with overwhelming probability by correctness of $\text{KEM}^\mathcal{L}$. For the remaining part of the proof, we assume that the decryption of the base-PKE scheme does not return $\perp$.¹¹ Next, we show that c decapsulates to k under $\overline{sk}^\mathcal{L}$ as well: As the underlying PKE scheme is restricted non-rigid, we have $\text{T}_\mathcal{P}[\text{PKE}, H_\top].\text{DEC}(\overline{sk}, c) = \perp$ (as the re-encryption check will fail with overwhelming probability) and hence the ciphertext c will be rejected under the secret key $\overline{sk}$, i.e., $\text{KEM}^\mathcal{L}.\text{DECAPS}(\overline{sk}^\mathcal{L}, c) = H_U(\overline{\sigma}, \cdot)$. This leads to

$$\text{KEM}^\mathcal{L}.\text{DECAPS}(sk^\mathcal{L}, c) = k = H_U(m, \cdot) = H_U(\overline{\sigma}, \cdot) = \text{KEM}^\mathcal{L}.\text{DECAPS}(\overline{sk}^\mathcal{L}, c)$$

which shows that $\mathcal{A}$ wins the game LEAK-BIND-K,CT-PK. Insecurity with respect to LEAK-BIND-K-PK, MAL-BIND-K,CT-PK, and MAL-BIND-K-PK follows from the hierarchy of the binding notions. $\square$

3.2 HON-BIND-K-PK Security for $\text{FO}^\mathcal{L}$

We establish a positive result for HON-BIND-K-PK security of $\text{FO}^\mathcal{L}$. More precisely, we show that—for $\text{FO}^\mathcal{L}$ —HON-BIND-K-PK is in fact equivalent to the generally weaker notion of HON-BIND-K,CT-PK. This effectively implies that [Theorem 1](#) also yields HON-BIND-K-PK security.

The following proposition establishes equivalence between HON-BIND-K-PK and HON-BIND-K,CT-PK for $\text{FO}^\mathcal{L}$ -KEMs. The difference between the two notions is that the former puts no restriction on the ciphertexts while the latter requires them to be the same. Since $\text{FO}^\mathcal{L}$ hashes the ciphertext to derive the shared key, we can conclude that the ciphertexts have to be different (unless there was a hash collision), effectively removing this freedom of the (generally stronger) notion HON-BIND-K-PK.

Proposition 1. *Let PKE be a PKE scheme and $\text{KEM}^\mathcal{L}$ the KEM obtained from applying $\text{FO}^\mathcal{L}$ to PKE. If $\text{KEM}^\mathcal{L}$ is X-BIND-K,CT-PK secure, then $\text{KEM}^\mathcal{L}$ is also X-BIND-K-PK secure for $X \in \{\text{HON}, \text{LEAK}\}$.*

¹¹Note that the decryption for KYBER.PKE never returns $\perp$, while for BIKE.PKE and HQC.PKE this is possible due to decoding failures.

Proof. We give the proof in terms of $X = \text{HON}$ and argue at the relevant parts what changes for $X = \text{LEAK}$. We show that if $\text{KEM}^\perp$ is not HON-BIND-K-PK secure, then it is also not HON-BIND-K,CT-PK secure. More precisely, given a HON-BIND-K-PK adversary $\mathcal{A}$, we construct a HON-BIND-K,CT-PK adversary $\mathcal{B}$ as follows: $\mathcal{B}$ obtains $(pk, \overline{pk})$, each of which is part of an honestly generated key pair $(pk, sk^\perp = (sk, \sigma)), (\overline{pk}, \overline{sk}^\perp = (\overline{sk}, \overline{\sigma}))$. $\mathcal{B}$ then calls the HON-BIND-K-PK adversary $\mathcal{A}$ against $\text{KEM}^\perp$ on input $(pk, \overline{pk})$ and simulates the decapsulation oracles for $\mathcal{A}$ using its own ones. When $\mathcal{A}$ outputs $(c, \bar{c})$, $\mathcal{B}$ in turn outputs (c, c) . For $X = \text{LEAK}$, $\mathcal{B}$ also gets the secret keys which $\mathcal{B}$ sends to $\mathcal{A}$; in this variant, there are no oracles that need to be simulated.

We assume that $\mathcal{A}$ wins the game HON-BIND-K-PK and show that then the adversary $\mathcal{B}$ as constructed above wins the game HON-BIND-K,CT-PK , except with negligible probability. As the keys are honestly generated, we can assume that $pk \neq \overline{pk}$ and $sk \neq \overline{sk}$, with overwhelming probability. Likewise, we can assume that $\sigma \neq \overline{\sigma}$ with overwhelming probability. $\mathcal{A}$ winning the game HON-BIND-K-PK implies that

$$k = \text{KEM}^\perp.\text{DECAPS}(sk^\perp, c) = \text{KEM}^\perp.\text{DECAPS}(\overline{sk}^\perp, \bar{c}) = \bar{k},$$

for c and $\bar{c}$ (not necessarily equal) output by $\mathcal{A}$. Let $m = \text{PKE}^\S.\text{DEC}(sk, c)$ and $\bar{m} = \text{PKE}^\S.\text{DEC}(\overline{sk}, \bar{c})$. There are three cases to consider, depending on whether the ciphertexts are valid or not. In all cases, the shared keys are computed as $k \leftarrow H_U(\cdot, c)$ and $\bar{k} \leftarrow H_U(\cdot, \bar{c})$; the mere difference is whether the first input is the decrypted message (m or $\bar{m}$) or the implicit rejection value (σ or $\overline{\sigma}$). But regardless of the first input—even in the LEAK case, when $\mathcal{A}$ knows both σ and $\overline{\sigma}$ —we can deduce $c = \bar{c}$ as otherwise $\mathcal{A}$ has found a collision for H_U . This yields that also $\mathcal{B}$ is successful in winning HON-BIND-K,CT-PK , which concludes the proof. $\square$

Note that the equivalence is limited to KEMs obtained via the $\text{FO}^\perp$ transform. In general, the two notions are not equivalent. Cremers et al. [CDM24] showed a separation between the notions for $\text{FO}_m^\perp$. Looking ahead, our results for $\text{FO}_m^\perp$ (cf. Corollary 1) also yield the separation between the notions HON-BIND-K-PK and HON-BIND-K,CT-PK as well as between the notions LEAK-BIND-K-PK and LEAK-BIND-K,CT-PK .

As we show that the notions HON-BIND-K-PK and HON-BIND-K,CT-PK are equivalent for $\text{FO}^\perp$ -KEMs in the above theorem, we can leverage Theorem 1 to show security wrt the notion HON-BIND-K-PK : if the underlying PKE scheme is SCFR-CPA secure, we get HON-BIND-K,CT-PK and thus HON-BIND-K-PK security for the $\text{FO}^\perp$ KEM. This is formalized in the following theorem.

Theorem 6. *Let PKE be a public-key encryption scheme that has negligible decryption errors and $\text{KEM}^\perp$ the key-encapsulation mechanism obtained from applying $\text{FO}^\perp$ to PKE. If $\text{PKE}^\S = \text{T}_\S[\text{PKE}, H_{\text{T}_\S}]$ is SCFR-CPA secure, then $\text{KEM}^\perp$ is HON-BIND-K,CT-PK secure.*

3.3 MAL-BIND-K-CT Security for $\text{FO}^\perp$

We show that key-encapsulation mechanisms obtained via the $\text{FO}^\perp$ transform are secure wrt the notions MAL-BIND-K-CT and MAL-BIND-K,PK-CT . The argument simply follows from the fact that the notions require the ciphertexts to be distinct while $\text{FO}^\perp$ derives the shared key by hashing also the ciphertext; any successful adversary therefore has to find a hash collision.

Theorem 7. *Let PKE be a public-key encryption scheme and $\text{KEM}^\perp$ the key-encapsulation mechanism obtained from applying $\text{FO}^\perp$ to PKE. Then $\text{KEM}^\perp$ is MAL-BIND-K-CT and MAL-BIND-K,PK-CT secure.*

Proof. Let $\mathcal{A}$ be an adversary against MAL-BIND-K-CT . Let c and $\bar{c}$ denote the ciphertexts related to $\mathcal{A}$'s output—which can either be direct outputs of $\mathcal{A}$ (in case of the DECAPS

variant) or obtained from an honest encapsulation using the public key and randomness output by $\mathcal{A}$ (in case of the ENCAPS variant). In order to win, these ciphertexts must be distinct, i.e., $c \neq \bar{c}$, while the encapsulated keys must agree, i.e., $k = \bar{k}$. However, by definition of FO^χ this yields $H_U(\cdot, c) = k = \bar{k} = H_U(\cdot, \bar{c})$ which means that $\mathcal{A}$ would have found a collision for H_U . Security wrt to MAL-BIND-K,PK-CT follows via the hierarchy of the notions. $\square$

While the results from [Theorem 7](#) were claimed in an earlier version of [\[CDM24\]¹²](#), the current version shows only LEAK-BIND-K,PK-CT. Our proof agrees with the one in the earlier version; we give it for sake of completeness but do not claim any novelty here.

3.4 MAL-BIND-K,CT-PK Attacks

We establish several negative results regarding MAL-BIND-K,CT-PK security of FO^χ as well as FO_m^χ . Given the hierarchy of the notions, this also establishes negative results with respect to MAL-BIND-K-PK and MAL-BIND-CT-PK for both FO^χ and FO_m^χ . Schmiege [\[Sch24\]](#) shows that ML-KEM is not MAL-BIND-K-PK secure which leaves the possibility that it might achieve MAL-BIND-K,CT-PK security (which is the generally weaker notion of the two). We show, that their attack also applies to MAL-BIND-K,CT-PK and for any KEM constructed via either FO^χ or FO_m^χ . In the attack, the adversary generates malicious secret keys that agree in their implicit rejection value. Then any invalid ciphertext (trivially obtained by randomly picking a ciphertext) results in the same shared key for both secret keys and thus breaks MAL-BIND-K,CT-PK security. This is formally stated in the theorem below.

Theorem 8. *Let PKE be a public-key encryption scheme and KEM be the key-encapsulation mechanism resulting from applying the implicitly-rejecting FO transform (regardless of which of the two variants) to PKE. Then, KEM is neither MAL-BIND-K,CT-PK nor MAL-BIND-K-PK secure.*

Proof. We give an MAL-BIND-K,CT-PK adversary $\mathcal{A}$ for variant (I) DECAPS-DECAPS. Adversary $\mathcal{A}$ first generates two key pairs for the underlying public-key encryption scheme, i.e., $(pk, sk), (\bar{pk}, \bar{sk}) \leftarrow \text{* KEYGEN}()$. Furthermore, $\mathcal{A}$ samples $\sigma \leftarrow \text{* } \mathcal{M}$ and sets $sk^\chi \leftarrow (sk, \sigma)$ as well as $\bar{sk}^\chi \leftarrow (\bar{sk}, \sigma)$. Finally, $\mathcal{A}$ generates an arbitrary ciphertext c , sets $\bar{c} \leftarrow c$ and outputs $(sk^\chi, \bar{sk}^\chi, c, \bar{c})$. The ciphertexts will be invalid (wrt both secret keys) with overwhelming probability¹³, hence $\text{KEM.DECAPS}(sk^\chi, c) = H_U(\sigma, c)$ and $\text{KEM.DECAPS}(\bar{sk}^\chi, c) = H_U(\sigma, c)$. This yields that $\mathcal{A}$ wins the game MAL-BIND-K,CT-PK. Insecurity wrt MAL-BIND-K-PK easily follows. $\square$

For ML-KEM and MAL-BIND-K-PK, the theorem above is exactly what is shown in [\[Sch24\]](#).

4 Application to BIKE and HQC

In this section, we analyze the binding properties of the code-based round-4 KEMs BIKE and HQC. We first provide a description of the schemes and the necessary background in [Section 4.1](#). Subsequently, in [Section 4.2](#), we analyze the binding properties of HQC and in [Section 4.3](#), we do the same for BIKE and HQC*, where the latter is a modified version of HQC that we introduce in [Section 4.1](#). While the modification is quite small, it results in improved binding security compared to the original HQC. Lastly, in [Section 4.4](#), we show

¹²The earlier version we are referring to is Version 1.0.6 which is available at <https://eprint.iacr.org/archive/2023/1933/20240403:091024>.

¹³Recall our assumption that the KEM is IND-CCA secure (cf. [Section 2.2](#)).

how our results can be transferred to the third round-4 KEM CLASSIC-MCELIECE and to ML-KEM to cover the last gaps left in the analysis of their binding properties.

4.1 Description of BIKE and HQC

BIKE [ABB⁺22] is a round-4 KEM based on Quasi-Cyclic Moderate Density Parity Check (QC-MDPC) codes. More precisely, BIKE is obtained by instantiating the Niederreiter scheme with QC-MDPC codes. Hence, the security can be traced back to the quasi-cyclic variants of certain distinguishing problems from coding theory. It results from applying the FO^χ transform to the public-key encryption scheme BIKE.PKE. Both BIKE.PKE and BIKE.KEM are displayed in Figure 5.

HQC [AAB⁺22] is a round-4 KEM based on quasi-cyclic codes and the hardness of the syndrome decoding problem. The public-key encryption scheme HQC.PKE and the key-encapsulation mechanism HQC.KEM, that results from applying a *variant of* the FO^χ transform to HQC.PKE, are shown in Figure 6. This variant introduces a random salt to protect against multi-ciphertext attacks. The salt is appended to the ciphertext, however, it is *not* part of the key derivation. Another change involving the salt is that the randomness for encryption is derived as $H_{T_s}(m, pk, salt)$ instead of $H_{T_s}(m)$. We describe the transformation used by HQC in Figure 7. This transformation relies on variants of T_s and U_s^χ , which we will refer to as T_s and U_s^χ , respectively, where the subscript indicates that they are randomized.

Note that this deviation has significant impact, as several results for the FO^χ transform no longer hold due to this change. This is why we will give a separate analysis for HQC in Section 4.2, covering a few positive but more negative results. We also consider a slightly different version of HQC, called HQC*, which achieves more binding properties. It differs from HQC in the fact that the salt is included in the final computation of the shared key (see Figure 6)—note that this is not simply the result of applying FO^χ to the base PKE scheme underlying HQC due to the salt.¹⁴ Nevertheless, this small modification allows us to apply the general results for the FO^χ transform, i.e., it suffices to check if the necessary prerequisites are fulfilled. The same is the case for BIKE, thus we analyze BIKE and HQC* together in Section 4.3.

Lastly, note that both BIKE and HQC sample certain key components using a distribution that is not completely uniform, but close to it. For BIKE this affects the secret key components h_0 and h_1 and analogously for HQC the secret key components $\mathbf{x}$ and $\mathbf{y}$. However, this does not have a relevant impact on security: for two games G and G' that differ only in the way the BIKE secret key is sampled—in G using a uniform distribution, in G' using the slightly biased one—the advantage of the adversary improves only by a factor τ . For the proposed parameter sets of BIKE [ABB⁺22, Section C.4] and HQC [AAB⁺24, Section 5.3], it holds that $\tau \leq 1.00353$. This is described in more detail in [ABB⁺22, Section C.4] and can be traced back to techniques by Sendrier [Sen11, Sen21]. For sake of simplicity, we make the assumption that $h_0, h_1, \mathbf{x}$, and $\mathbf{y}$ are uniformly sampled for the following section and omit the factor τ from the bounds.¹⁵

4.2 Binding Security of HQC

In this section, we analyze the binding security of HQC. Since it is an implicitly-rejecting KEM, there are 12 notions to be considered, which are depicted in Figure 1. As described in the previous section, HQC does not apply the standard FO^χ transform. More precisely, for HQC a ciphertext contains a salt value, which (together with the message and public

¹⁴IND-CCA security for the variant of the FO transform used by HQC* follows from [HHK17, Theorem 3.2 and Theorem 3.4]—in fact, this variant is closer to the original FO transform, as it hashes the entire ciphertext which the variant used in HQC does not.

¹⁵We use this assumption in Proposition 3.

BIKE.PKE.KEYGEN()	BIKE.PKE.ENC(pk, m)	BIKE.PKE.DEC(sk, c)
$(h_0, h_1) \leftarrow \mathcal{H}_w$	$(e_0, e_1) \leftarrow \mathcal{E}_t$	$\bar{e} \leftarrow \text{DECODER}(c_0 h_0, h_0, h_1)$
$h \leftarrow h_0 h_1^{-1}$	$c_0 \leftarrow e_0 + e_1 h$	if $\bar{e} = \perp$
$pk \leftarrow h$	$c_1 \leftarrow m \oplus \mathbf{H}_L(e_0, e_1)$	$m \leftarrow \perp$
$sk \leftarrow (h_0, h_1)$	$c \leftarrow (c_0, c_1)$	else
return (pk, sk)	return c	$m \leftarrow c_1 \oplus \mathbf{H}_L(\bar{e})$
		return m

BIKE.KEYGEN()	BIKE.ENCAPS(pk)	BIKE.DECAPS(sk, c)
$(h_0, h_1) \leftarrow \mathcal{H}_w$	$m \leftarrow \mathcal{M}$	$\bar{e} \leftarrow \text{DECODER}(c_0 h_0, h_0, h_1)$
$h \leftarrow h_0 h_1^{-1}$	$(e_0, e_1) \leftarrow \mathbf{H}_H(m)$	$\bar{m} \leftarrow c_1 \oplus \mathbf{H}_L(\bar{e})$
$\sigma \leftarrow \mathcal{M}$	$c_0 \leftarrow e_0 + e_1 h$	if $\bar{e} = \mathbf{H}_H(\bar{m})$
$pk \leftarrow h$	$c_1 \leftarrow m \oplus \mathbf{H}_L(e_0, e_1)$	$k \leftarrow \mathbf{H}_U(\bar{m}, c)$
$sk \leftarrow (h_0, h_1, \sigma)$	$c \leftarrow (c_0, c_1)$	else
return (pk, sk)	$k \leftarrow \mathbf{H}_U(m, c)$	$k \leftarrow \mathbf{H}_U(\sigma, c)$
	return (c, k)	return k

Figure 5: PKE scheme BIKE.PKE and key-encapsulation mechanism BIKE. Here, $\mathcal{H}_w = \{(h_0, h_1) \in \mathcal{R}^2 \mid |h_0| = |h_1| = w/2\}$ and $\mathcal{E}_t = \{(e_0, e_1) \in \mathcal{R}^2 \mid |e_0| + |e_1| = t\}$ for $\mathcal{R} = \mathbb{F}_2[X]/(X^r - 1)$ for the BIKE parameters w, r and t . Note that $\mathbf{H}_H, \mathbf{H}_L$, and $\mathbf{H}_U$ are hash functions.

key) is used to derive the randomness for the re-encryption approach. While the general attacks we give for $\text{FO}^\mathcal{X}$ -KEMs are still applicable¹⁶, the general proofs from Section 2 and Section 3 cannot be transferred this easily. It turns out that HQC only achieves 2 out of the 12 binding notions: HON-BIND-K-PK and HON-BIND-K,CT-PK. Note that the notions X-BIND-K-CT and X-BIND-K,PK-CT (for $X \in \{\text{LEAK}, \text{MAL}\}$) are achieved by *any* $\text{FO}^\mathcal{X}$ -KEM, thus the modification of the $\text{FO}^\mathcal{X}$ transform made by HQC negatively affects the results for the binding properties.

4.2.1 Attacking MAL/LEAK Binding Notions for HQC.

Firstly, all attacks against X-BIND-K-PK and X-BIND-K,CT-PK for $X \in \{\text{LEAK}, \text{MAL}\}$ still apply to HQC. This is formulated in the following theorem which is proven in Appendix C.1.

Theorem 9. *The key-encapsulation mechanism HQC as shown in Figure 6 is insecure with respect to the following binding notions: LEAK-BIND-K,CT-PK, LEAK-BIND-K-PK, MAL-BIND-K,CT-PK, and MAL-BIND-K-PK.*

The following theorem (proven in Appendix C.2) proves HQC to be insecure with respect to the notion LEAK-BIND-K,PK-CT, which is contrary to the results for $\text{FO}^\mathcal{X}$. From this, a number of other LEAK/MAL attacks follow. The overall attack idea follows the strategy of our attack against LEAK-BIND-K,CT-PK for $\text{FO}^\mathcal{X}$ -KEMs (cf. Theorem 5): We construct an honest ciphertext—using the implicit rejection value as the random message—and create a second, invalid ciphertext by changing the salt value of the ciphertext. Since the salt is not used to derive the key k , the valid and invalid ciphertexts result in the same key.

¹⁶These results cover the general notions X-BIND-K-PK and X-BIND-K,CT-PK for $X \in \{\text{LEAK}, \text{MAL}\}$. The attacks described for the $\text{FO}^\mathcal{X}$ transform still apply by choosing the salts (contained in the ciphertexts that are output by the adversary) to be equal.

HQC.PKE.KEYGEN()	HQC.PKE.ENC(pk, m)	HQC.PKE.DEC(sk, c)
$\mathbf{h} \leftarrow \mathcal{R}$	$(\mathbf{e}, \mathbf{r}_1, \mathbf{r}_2) \leftarrow \mathcal{R}_{w_e} \times \mathcal{R}_{w_r} \times \mathcal{R}_{w_r}$	$m \leftarrow \text{DECODER}(\mathbf{v} - \mathbf{u}\mathbf{y})$
$\mathbf{G} \leftarrow \mathbb{F}_2^{k \times n}$	$\mathbf{u} \leftarrow \mathbf{r}_1 + \mathbf{h}\mathbf{r}_2$	return m
$\mathbf{x}, \mathbf{y} \leftarrow \mathcal{R}_w \times \mathcal{R}_w$	$\mathbf{v} \leftarrow \text{TRUNC}(m\mathbf{G} + \mathbf{s}\mathbf{r}_2 + \mathbf{e}, \ell)$	
$\mathbf{s} \leftarrow \mathbf{x} + \mathbf{h}\mathbf{y}$	$c \leftarrow (\mathbf{u}, \mathbf{v})$	
$pk \leftarrow (\mathbf{h}, \mathbf{s})$	return c	
$sk \leftarrow (\mathbf{x}, \mathbf{y})$		
return (pk, sk)		
HQC.KEYGEN()	HQC.ENCAPS(pk)	HQC.DECAPS(sk, c)
$\mathbf{h} \leftarrow \mathcal{R}$	$m \leftarrow \mathbb{F}_2^k$	$\bar{m} \leftarrow \text{DECODER}(\mathbf{v} - \mathbf{u}\mathbf{y})$
$\mathbf{G} \leftarrow \mathbb{F}_2^{k \times n}$	$salt \leftarrow \mathbb{F}_2^{128}$	$(\bar{\mathbf{e}}, \bar{\mathbf{r}}_1, \bar{\mathbf{r}}_2) \leftarrow H_{\mathcal{T}_g}(\bar{m}, pk, salt)$
$\mathbf{x}, \mathbf{y} \leftarrow \mathcal{R}_w \times \mathcal{R}_w$	$(\mathbf{e}, \mathbf{r}_1, \mathbf{r}_2) \leftarrow H_{\mathcal{T}_g}(m, pk, salt)$	$\bar{\mathbf{u}} \leftarrow \bar{\mathbf{r}}_1 + \mathbf{h}\bar{\mathbf{r}}_2$
$\sigma \leftarrow \mathcal{M}$	$\mathbf{u} \leftarrow \mathbf{r}_1 + \mathbf{h}\mathbf{r}_2$	$\bar{\mathbf{v}} \leftarrow \text{TRUNC}(\bar{m}\mathbf{G} + \mathbf{s}\bar{\mathbf{r}}_2 + \bar{\mathbf{e}}, \ell)$
$\mathbf{s} \leftarrow \mathbf{x} + \mathbf{h}\mathbf{y}$	$\mathbf{v} \leftarrow \text{TRUNC}(m\mathbf{G} + \mathbf{s}\mathbf{r}_2 + \mathbf{e}, \ell)$	if $(\bar{\mathbf{u}}, \bar{\mathbf{v}}) = (\mathbf{u}, \mathbf{v})$
$pk \leftarrow (\mathbf{h}, \mathbf{s})$	$k \leftarrow H_U(m, (\mathbf{u}, \mathbf{v}, salt))$	$k \leftarrow H_U(\bar{m}, (\mathbf{u}, \mathbf{v}, salt))$
$sk \leftarrow (\mathbf{x}, \mathbf{y}, \sigma)$	$c \leftarrow (\mathbf{u}, \mathbf{v}, salt)$	else
return (pk, sk)	return (c, k)	$k \leftarrow H_U(\sigma, (\mathbf{u}, \mathbf{v}, salt))$
		return k

Figure 6: PKE scheme HQC.PKE and key-encapsulation mechanism HQC. Here, $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$ and $\mathcal{R}_x = \{v \in \mathcal{R} \text{ with hamming weight } x\}$ for $x \in \{w, w_e, w_r\}$; the latter are HQC parameters. Note that, H_U is a hash function. The modified version HQC* includes the highlighted parts, i.e., the *entire* ciphertext is hashed to compute the key k .

Theorem 10. *The key-encapsulation mechanism HQC as shown in Figure 6 is neither X-BIND-K,PK-CT nor X-BIND-K-CT secure for $X \in \{\text{MAL}, \text{LEAK}\}$.*

4.2.2 Proving/Attacking HON Binding Notions for HQC.

The theorem below, which is proven in Appendix C.3, shows that HQC achieves both HON-BIND-K-PK and HON-BIND-K,CT-PK.

Theorem 11. *The key-encapsulation mechanism HQC as displayed in Figure 6 fulfills the notions HON-BIND-K-PK and HON-BIND-K,CT-PK.*

The following theorem shows that HQC does not achieve HON-BIND-K-CT and HON-BIND-K,PK-CT. The proof can be found in Appendix C.4. The attack exploits the fact that for two invalid ciphertexts that differ only in the salt, the same shared key will be computed if the same key pair is used.

Theorem 12. *The KEM HQC as displayed in Figure 6 is not HON-BIND-K-CT and HON-BIND-K,PK-CT secure.*

4.3 Binding Security of BIKE and HQC*

The fact that BIKE uses $\text{FO}^\perp$ already provides several results for different binding notions. While HQC* does not use FO , its variant is close enough that the same results apply. However, for LEAK-BIND-K,CT-PK, LEAK-BIND-K-PK, HON-BIND-K,CT-PK, and HON-BIND-K,CT-PK our results rely on additional assumptions. To show an attack against the former two notions, we rely on the restricted non-rigidity property while

$\text{PKE}^{\$}.\text{KEYGEN}()$	$\text{PKE}^{\$}.\text{ENC}(pk, m)$	$\text{PKE}^{\$}.\text{DEC}(sk, (c_*, salt))$
$(pk, sk) \leftarrow \text{KEYGEN}()$	$salt \leftarrow \mathcal{S}$	$m \leftarrow \text{DEC}(sk, c_*)$
return (pk, sk)	$r \leftarrow \text{H}_T(m, pk, salt)$	$r \leftarrow \text{H}_T(m, pk, salt)$
	$c_* \leftarrow \text{ENC}(pk, m; r)$	$\bar{c}_* \leftarrow \text{ENC}(pk, m; r)$
	return $(c_*, salt)$	if $\bar{c}_* \neq c_*$
		return $\perp$
		return m

$\text{KEYGEN}^{\mathcal{X}}()$	$\text{ENCAPS}(pk)$	$\text{DECAPS}(sk^{\mathcal{X}}, (c_*, salt))$
$(pk, sk) \leftarrow \text{KEYGEN}()$	$m \leftarrow \mathcal{M}$	$(sk, \sigma) \leftarrow sk^{\mathcal{X}}$
$\sigma \leftarrow \mathcal{M}$	$(c_*, salt) \leftarrow \text{PKE}^{\$}.\text{ENC}(pk, m)$	$m \leftarrow \text{PKE}^{\$}.\text{DEC}(sk, (c_*, salt))$
$sk^{\mathcal{X}} \leftarrow (sk, \sigma)$	$k \leftarrow \text{H}_U(m, c_*)$	if $m \neq \perp$
return $(pk, sk^{\mathcal{X}})$	return $((c_*, salt), k)$	return $\text{H}_U(m, c_*)$
		return $\text{H}_U(\sigma, c_*)$

Figure 7: Transformations used by HQC. **Top:** The PKE scheme $\text{PKE}^{\$} = \mathcal{T}_{\$}[\text{PKE}]$. **Bottom:** The KEM $\text{U}_{\$}^{\mathcal{X}}[\mathcal{T}_{\$}[\text{PKE}]]$.

security with respect to the latter two requires the underlying $\mathcal{T}_{\mathcal{G}}$ -transformed PKE scheme to achieve SCFR-CPA. The two propositions below establish these properties for BIKE and HQC*. Note that the $\mathcal{T}_{\$}$ -transformed PKEs underlying HQC and HQC* agree (we will use HQC.PKE as notation for both), i.e., the below results apply for both schemes.

The following proposition is proven in [Appendix C.5](#).

Proposition 2. *Let PKE be a public-key encryption scheme with the following property: for public keys $pk, \bar{pk}$ stemming from honest key generations, two (not necessarily different) messages $m, \bar{m}$, and two random coins $r, \bar{r}$, the ciphertexts $c = \text{PKE}.\text{ENC}(pk, m; r)$ and $\bar{c} = \text{PKE}.\text{ENC}(\bar{pk}, \bar{m}; \bar{r})$ differ with overwhelming probability. Then the following statements hold:*

1. The PKE scheme $\mathcal{X}[\text{PKE}, \text{H}_T]$ is SCFR-LEAK secure for $\mathcal{X} \in \{\mathcal{T}_{\mathcal{G}}, \mathcal{T}_{\$}\}$.
2. The PKE scheme PKE is restricted non-rigid.

The proof for the proposition below can be found in [Appendix C.6](#).

Proposition 3. *Consider the public-key encryption schemes BIKE.PKE and HQC.PKE as shown in [Figure 5](#) and [Figure 6](#), respectively. The following statements hold:*

1. The PKE scheme $\mathcal{T}_{\mathcal{G}}[\text{BIKE.PKE}, \text{H}_T]$ is SCFR-LEAK secure.
2. BIKE.PKE fulfills restricted non-rigidity.
3. The PKE scheme $\mathcal{T}_{\$}[\text{HQC.PKE}, \text{H}_T]$ is SCFR-LEAK secure.
4. HQC.PKE fulfills restricted non-rigidity.

Having established [Proposition 2](#) and [Proposition 3](#), we get the following theorem regarding the binding properties of BIKE and HQC*—the results are exactly those that are presented for $\text{FO}^{\mathcal{X}}$ in [Table 1](#). Note that HQC* deviates from the standard $\text{FO}^{\mathcal{X}}$ transform only in the way the randomness is derived. However, one can easily check that this change is irrelevant for our $\text{FO}^{\mathcal{X}}$ results. Due to this and the fact that all requirements are fulfilled, we can apply our $\text{FO}^{\mathcal{X}}$ results to HQC*.

Theorem 13. *The key-encapsulation mechanisms BIKE and HQC* as shown in Figure 5 and Figure 6 are HON-BIND-K,CT-PK and HON-BIND-K-PK as well as X-BIND-K-CT and X-BIND-K,PK-CT secure for $X \in \{\text{HON}, \text{LEAK}, \text{MAL}\}$. They are insecure wrt the notions LEAK-BIND-K,CT-PK, LEAK-BIND-K-PK, MAL-BIND-K,CT-PK, and MAL-BIND-K-PK.*

4.4 Binding Security of Round-4 KEMs and ML-KEM

In the previous sections, we completed the analysis of the binding properties of the key-encapsulation mechanisms BIKE, HQC, and HQC*. Furthermore, our results cover the last gaps left in the analysis of CLASSIC-McELIECE and ML-KEM: Theorem 7 proves CLASSIC-McELIECE to be MAL-BIND-K-CT secure. Further, ML-KEM is not MAL-BIND-K,CT-PK secure by Theorem 8 (the attack relies exclusively on invalid ciphertexts for which ML-KEM behaves like $\text{FO}^\perp$) while it is MAL-BIND-K,PK-CT secure by Theorem 18 with the following changes to the proof. Case 1 (both ciphertexts are invalid) works exactly the same way. Case 2 (one ciphertext is invalid) is different in two aspects: (1) for the valid ciphertext, the shared key is computed as the hash of *both* message and public key; (2) different hash functions are used to compute k and $\bar{k}$. Nevertheless, one can see that an adversary needs to find a collision to be successful. Case 3 (both ciphertexts are valid) works essentially the same: the difference is that the shared keys are computed as the hash of message and public key (compared to just the message), however, this does not matter for the conclusion that the messages are equal.

The completed results regarding the binding properties of the round-4 KEMs (and ML-KEM) can be found in Table 2.

5 Achieving Binding Security

In this section, we present ways to achieve binding security for implicitly-rejecting KEMs. In Section 5.1, we describe transforms for the different attack models HON, LEAK, and MAL, while the corresponding results are given in Section 5.2.

5.1 Description of the Transforms FO_H , FO_L , and FO_M

In the following, we describe how different levels of binding security can be achieved in general for implicitly-rejecting KEMs—an overview is given in Table 3. For this, we take into account the general results presented in Section 3 as well as conclusions that can be drawn from the scheme-specific analysis in Section 4. We want to highlight the fact that if we talk about transforms achieving *all* HON/LEAK/MAL binding notions in this section, we mean all notions that are generally possible for implicitly-rejecting KEMs—note that there are further ones, that can never be achieved for such schemes (namely X-BIND-CT-PK and X-BIND-CT-K).

All transforms are variants of the FO transform. More precisely, they are the composition of the T transform and a variant of the U transform; in our description, we focus on the latter. The full transforms are dubbed FO_H , FO_L , and FO_M , the subscript indicating which security level each transform achieves.

FO_H (**accept:** $k = H_U(m, c)$; **reject:** $k = H_U(\sigma, c)$).

We observe that all HON notions are fulfilled by KEMs using the $\text{FO}^\perp$ transform (Table 1).¹⁷ Henceforth we refer to this as the FO_H transform. In the LEAK setting, this transform fails to achieve the notions LEAK-BIND-K-PK and LEAK-BIND-K,CT-PK; a natural

¹⁷Note that some results depend on a mild assumption.

way to also attain these notions would be to additionally hash the public key. The results for ML-KEM, however, indicate that one can obtain all LEAK notions by hashing the public key *instead* of the ciphertext in the accepting case.

FO_L (**accept:** $k = H_U(m, h_{pk})$; **reject:** $k = H_U(\sigma, c)$).

We show that the variant of the FO transform used in ML-KEM—to be precise the variant of the U transform which derives the shared key from the message and hash of the public key—achieves all LEAK notions and we will refer to this one as the FO_L transform. This change prevents our LEAK attack (cf. [Theorem 5](#)) by using different inputs in the accepting and rejecting case. The FO_L transform, however, fails to achieve some of the MAL notions.

FO_M (**accept:** $k = H_U(m, h_{pk})$; **reject:** $k = H_U(\sigma, h_{pk}, c)$).

We finally describe a transform (FO_M) that achieves all MAL notions and thus all binding notions in general. A simple approach would be to ensure that everything (message, ciphertext, and public key) is hashed to derive the shared key k —in the rejecting case, the implicit rejection value is used rather than the message. Surprisingly, we show that this is not necessary: In the accepting case, it is sufficient to hash *only* the message and the public key. More precisely, the MAL transform can be considered as a modified variant of the FO_m^ℓ transform, where the public key is added to the final hash computation both in the accepting and rejecting case—equivalently, one can view it as the FO_L transform enhanced by hashing the public key also in the rejection case. This keeps the benefit that the ciphertext does not have to be hashed for valid ciphertexts, which would be much more costly.

5.1.1 Comparison of the Variants.

We want to highlight the fact that the FO_H transform will (in most cases) be more costly than the FO_L transform due to hashing the ciphertext rather than the hash of the public key. Thus, when deciding on which of the binding transforms to use, it makes sense to choose FO_L over FO_H, even if the use-case at hand requires only HON-security. In particular, as many schemes already deploy the FO_m^ℓ transform (and thus FO_H), a switch to FO_L is neither very invasive nor produces overhead, but brings the benefits of more binding security. Regarding the cost of FO_M, the transform is as efficient as FO_L in the accepting case and only slightly more expensive in the rejecting case (additional hashing of $h_{pk} = H(pk)$). In view of this, FO_M seems like a good choice as it achieves all binding notions at a small efficiency loss compared to FO_m^ℓ.

5.1.2 IND-CCA Security of the Transforms.

Note that the three transforms FO_H, FO_L, and FO_M preserve IND-CCA security. For FO_H, this follows directly from prior results for the FO_m^ℓ transform. FO_L is the variant of the FO transform used in ML-KEM, which is proven IND-CCA secure—hence IND-CCA security is inherited from the NIST standard. Lastly, FO_M differs from FO_L only by additionally hashing the public key in the rejection case. Since we just add a public value to the key derivation, this does not affect security, which relies on the implicit rejection value being unknown to the adversary.

5.1.3 A Note on the Hash of the Public Key.

The hash of the public key is often added to the secret key to avoid hashing it for each key decapsulation at the cost of a slightly larger key. This approach, however, opens the possibility for malicious keys to not contain the actual hash value but some maliciously

chosen value. Such attacks are described by Schmieg [Sch24]. Thus, the secret key needs to be stored securely so that an adversary can neither read-out nor tamper with it. This way, the remaining attack vectors arise from malicious implementations, e.g., one where the hash of the public key is incorrectly computed. This threat can be mitigated by explicitly hashing the public key and comparing the result with the hash value that is part of the secret key. Note that this needs to be done only once, right after generating the key-pair. Together with our transform FO_M , one can then achieve full MAL-BIND security.

5.2 LEAK/MAL Security for FO_L/FO_M

The following theorem proves LEAK security of an FO_L -transformed KEM. More precisely, we show that the notions LEAK-BIND-K-PK and LEAK-BIND-K-CT are fulfilled, which then implies that all LEAK and HON notions hold (by the hierarchy visualized in Figure 1). The proof is given in Appendix C.7.

Theorem 14. *Let PKE be a PKE scheme and KEM the KEM obtained from applying FO_L to PKE. Then KEM is X-BIND-K-PK, X-BIND-K-CT, X-BIND-K,CT-PK, and X-BIND-K,PK-CT secure for $X \in \{\text{LEAK}, \text{HON}\}$.*

The next theorem shows that the FO_M transform fulfills MAL-BIND-K-PK and MAL-BIND-K-CT security and hence—by the hierarchy—all binding notions. The proof can be found in Appendix C.8.

Theorem 15. *Let PKE be a PKE scheme and KEM the KEM obtained from applying FO_M to PKE. Then KEM is X-BIND-K-PK, X-BIND-K-CT, X-BIND-K,CT-PK, and X-BIND-K,PK-CT secure for $X \in \{\text{MAL}, \text{LEAK}, \text{HON}\}$.*

References

- [AAB⁺20] Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, and Jurjen Bos. HQC. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [AAB⁺22] Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, Arnaud Dion, Jerome Lacan, Jean-Marc Robert, and Pascal Veron. HQC. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-4-submissions>.
- [AAB⁺24] Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, Arnaud Dion, Jerome Lacan, Jean-Marc Robert, and Pascal Veron. HQC. Technical report, NIST, 2024. available at https://pqc-hqc.org/doc/hqc-specification_2024-02-23.pdf.
- [ABB⁺22] Nicolas Aragon, Paulo Barreto, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Phillipe Gaborit, Shay Gueron, Tim Guneyasu, Carlos Aguilar-Melchor, Rafael Misoczki, Edoardo Persichetti, Nicolas Sendrier, Jean-Pierre Tillich, Gilles Zémor, Valentin Vasseur, Santosh Ghosh, and Jan Richter-Brokmann. BIKE. Technical report, National Institute of Standards

- and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-4-submissions>.
- [ABC⁺20] Martin R. Albrecht, Daniel J. Bernstein, Tung Chou, Carlos Cid, Jan Gilcher, Tanja Lange, Varun Maram, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Kenneth G. Paterson, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, Jakub Szefer, Cen Jung Tjhai, Martin Tomlinson, and Wen Wang. Classic McEliece. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [ABC⁺22] Martin R. Albrecht, Daniel J. Bernstein, Tung Chou, Carlos Cid, Jan Gilcher, Tanja Lange, Varun Maram, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Kenneth G. Paterson, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, Jakub Szefer, Cen Jung Tjhai, Martin Tomlinson, and Wen Wang. Classic McEliece. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-4-submissions>.
- [ABN10] Michel Abdalla, Mihir Bellare, and Gregory Neven. Robust encryption. In Daniele Micciancio, editor, *TCC 2010*, volume 5978 of *LNCS*, pages 480–497. Springer, Berlin, Heidelberg, February 2010. doi:10.1007/978-3-642-11799-2_28.
- [ACP⁺19] Martin Albrecht, Carlos Cid, Kenneth G. Paterson, Cen Jung Tjhai, and Martin Tomlinson. NTS-KEM. Technical report, National Institute of Standards and Technology, 2019. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-2-submissions>.
- [ADG⁺22] Ange Albertini, Thai Duong, Shay Gueron, Stefan Kölbl, Atul Luykx, and Sophie Schmieg. How to abuse and fix authenticated encryption without key commitment. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022*, pages 3291–3308. USENIX Association, August 2022. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/albertini>.
- [ADM⁺24] Thomas Aulbach, Samed Düzl , Michael Meyer, Patrick Struck, and Maximiliane Weish upl. Hash your keys before signing - BUFF security of the additional NIST PQC signatures. In Markku-Juhani Saarinen and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part II*, pages 301–335. Springer, Cham, June 2024. doi:10.1007/978-3-031-62746-0_13.
- [AHK⁺23] Jo l Alwen, Dominik Hartmann, Eike Kiltz, Marta Mularczyk, and Peter Schwabe. Post-quantum multi-recipient public key encryption. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 1108–1122. ACM Press, November 2023. doi:10.1145/3576915.3623185.
- [Aye15] Andrew Ayer. Duplicate signature key selection attack in Let’s Encrypt. https://www.agwa.name/blog/post/duplicate_signature_key_selection_attack_in_lets_encrypt, 2015.

- [BBC⁺19] Marco Baldi, Alessandro Barengi, Franco Chiaraluce, Gerardo Pelosi, and Paolo Santini. LEDAcrypt. Technical report, National Institute of Standards and Technology, 2019. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-2-submissions>.
- [BBDP01] Mihir Bellare, Alexandra Boldyreva, Anand Desai, and David Pointcheval. Key-privacy in public-key encryption. In Colin Boyd, editor, *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 566–582. Springer, Berlin, Heidelberg, December 2001. doi:10.1007/3-540-45682-1_33.
- [BCD⁺24] Manuel Barbosa, Deirdre Connolly, João Diogo Duarte, Aaron Kaiser, Peter Schwabe, Karolin Varner, and Bas Westerbaan. X-wing. *IACR Communications in Cryptology*, 1(1), 2024. doi:10.62056/a3qj89n4e.
- [BDK⁺18] Joppe W. Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS - kyber: A CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy*, pages 353–367. IEEE Computer Society Press, April 2018. doi:10.1109/EuroSP.2018.00032.
- [BH22] Mihir Bellare and Viet Tung Hoang. Efficient schemes for committing authenticated encryption. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 845–875. Springer, Cham, May / June 2022. doi:10.1007/978-3-031-07085-3_29.
- [BH24] Mihir Bellare and Viet Tung Hoang. Succinctly-committing authenticated encryption. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part IV*, volume 14923 of *LNCS*, pages 305–339. Springer, Cham, August 2024. doi:10.1007/978-3-031-68385-5_10.
- [BP18] Daniel J. Bernstein and Edoardo Persichetti. Towards KEM unification. Cryptology ePrint Archive, Report 2018/526, 2018. URL: <https://eprint.iacr.org/2018/526>.
- [CDF⁺21] Cas Cremers, Samed Düzlül, Rune Fiedler, Marc Fischlin, and Christian Janson. BUFFing signature schemes beyond unforgeability and the case of post-quantum signatures. In *2021 IEEE Symposium on Security and Privacy*, pages 1696–1714. IEEE Computer Society Press, May 2021. doi:10.1109/SP40001.2021.00093.
- [CDM24] Cas Cremers, Alexander Dax, and Niklas Medinger. Keeping up with the KEMs: Stronger security notions for KEMs and automated analysis of KEM-based protocols. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 1046–1060. ACM Press, October 2024. doi:10.1145/3658644.3670283.
- [Den03] Alexander W. Dent. A designer’s guide to KEMs. In Kenneth G. Paterson, editor, *9th IMA International Conference on Cryptography and Coding*, volume 2898 of *LNCS*, pages 133–151. Springer, Berlin, Heidelberg, December 2003. doi:10.1007/978-3-540-40974-8_12.
- [DFF24] Samed Düzlül, Rune Fiedler, and Marc Fischlin. BUFFing FALCON without increasing the signature size. In Maria Eichlseder and Sébastien Gambs, editors, *SAC 2024, Part I*, volume 15516 of *LNCS*, pages 131–150. Springer, Cham, August 2024. doi:10.1007/978-3-031-82852-2_6.

- [DFH⁺24] Jelle Don, Serge Fehr, Yu-Hsuan Huang, Jyun-Jie Liao, and Patrick Struck. Hide-and-seek and the non-resignability of the BUFF transform. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part III*, volume 15366 of *LNCS*, pages 347–370. Springer, Cham, December 2024. doi:10.1007/978-3-031-78020-2_12.
- [DFHS24] Jelle Don, Serge Fehr, Yu-Hsuan Huang, and Patrick Struck. On the (in)security of the BUFF transform. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part I*, volume 14920 of *LNCS*, pages 246–275. Springer, Cham, August 2024. doi:10.1007/978-3-031-68376-3_8.
- [DGRW18] Yevgeniy Dodis, Paul Grubbs, Thomas Ristenpart, and Joanne Woodage. Fast message franking: From invisible salamanders to encryptment. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 155–186. Springer, Cham, August 2018. doi:10.1007/978-3-319-96884-1_6.
- [DKR⁺20] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, Frederik Vercauteren, Jose Maria Bermudo Mera, Michiel Van Beirendonck, and Andrea Basso. SABER. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [FG25] Rune Fiedler and Felix Günther. Security analysis of Signal’s PQXDH handshake. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025, Part II*, volume 15675 of *LNCS*, pages 137–169. Springer, Cham, May 2025. doi:10.1007/978-3-031-91823-0_5.
- [FO99] Eiichiro Fujisaki and Tatsuki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages 537–554. Springer, Berlin, Heidelberg, August 1999. doi:10.1007/3-540-48405-1_34.
- [GMP22] Paul Grubbs, Varun Maram, and Kenneth G. Paterson. Anonymous, robust post-quantum public key encryption. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 402–432. Springer, Cham, May / June 2022. doi:10.1007/978-3-031-07082-2_15.
- [HHK17] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 341–371. Springer, Cham, November 2017. doi:10.1007/978-3-319-70500-2_12.
- [JCCS19] Dennis Jackson, Cas Cremers, Katriel Cohn-Gordon, and Ralf Sasse. Seems legit: Automated analysis of subtle attacks on protocols that use signatures. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 2165–2180. ACM Press, November 2019. doi:10.1145/3319535.3339813.
- [KSW24] Juliane Krämer, Patrick Struck, and Maximiliane Weishäupl. Committing AE from sponges security analysis of the NIST LWC finalists. *IACR Trans. Symm. Cryptol.*, 2024(4):191–248, 2024. doi:10.46586/tosc.v2024.i4.191-248.

- [LGR21] Julia Len, Paul Grubbs, and Thomas Ristenpart. Partitioning oracle attacks. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 195–212. USENIX Association, August 2021. URL: <https://www.usenix.org/conference/usenixsecurity21/presentation/len>.
- [MLGR23] Sanketh Menda, Julia Len, Paul Grubbs, and Thomas Ristenpart. Context discovery and commitment attacks - how to break CCM, EAX, SIV, and more. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part IV*, volume 14007 of *LNCS*, pages 379–407. Springer, Cham, April 2023. doi:10.1007/978-3-031-30634-1_13.
- [NAB⁺20] Michael Naehrig, Erdem Alkim, Joppe Bos, Léo Ducas, Karen Easterbrook, Brian LaMacchia, Patrick Longa, Ilya Mironov, Valeria Nikolaenko, Christopher Peikert, Ananth Raghunathan, and Douglas Stebila. FrodoKEM. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [NIS17] NIST. Post-quantum cryptography standardization process. <https://csrc.nist.gov/projects/post-quantum-cryptography>, 2017.
- [NIS22] NIST. Call for additional digital signature schemes for the post-quantum cryptography standardization process. <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/call-for-proposals-dig-sig-sep-t-2022.pdf>, 2022.
- [NIS23] NIST. Module-Lattice-based Key-Encapsulation Mechanism Standard. <https://doi.org/10.6028/NIST.FIPS.203.ipd>, 2023. Draft.
- [NIS24] NIST. Accordion mode. <https://csrc.nist.gov/pubs/other/2024/04/10/proposal-of-requirements-for-an-accordion-mode-dis/iprd>, 2024.
- [SAB⁺20] Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, and Damien Stehlé. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [Sch24] Sophie Schmieg. Unbindable kemmy schmidt: ML-KEM is neither MAL-BIND-K-CT nor MAL-BIND-K-PK. Cryptology ePrint Archive, Report 2024/523, 2024. URL: <https://eprint.iacr.org/2024/523>.
- [Sen11] Nicolas Sendrier. Decoding one out of many. In Bo-Yin Yang, editor, *Post-Quantum Cryptography - 4th International Workshop, PQCrypto 2011*, pages 51–67. Springer, Berlin, Heidelberg, November / December 2011. doi:10.1007/978-3-642-25405-5_4.
- [Sen21] Nicolas Sendrier. Secure sampling of constant-weight words – application to BIKE. Cryptology ePrint Archive, Report 2021/1631, 2021. URL: <https://eprint.iacr.org/2021/1631>.
- [SW24] Patrick Struck and Maximiliane Weishäupl. Constructing committing and leakage-resilient authenticated encryption. *IACR Trans. Symm. Cryptol.*, 2024(1):497–528, 2024. doi:10.46586/tosc.v2024.i1.497-528.

- [Xag22] Keita Xagawa. Anonymity of NIST PQC round 3 KEMs. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 551–581. Springer, Cham, May / June 2022. doi:10.1007/978-3-031-07082-2_20.

A Additional Background

Figure 8 and Figure 9 display the various binding notions considered in this work.

Game HON-BIND-K,CT-PK	Game MAL-BIND-K,CT-PK
$(pk, sk) \leftarrow \text{KEYGEN}()$ $(\overline{pk}, \overline{sk}) \leftarrow \text{KEYGEN}()$ $(c, \overline{c}) \leftarrow \mathcal{A}^{\text{Dec}, \overline{\text{Dec}}}(pk, \overline{pk})$ $k \leftarrow \text{DECAPS}(sk, c)$ $\overline{k} \leftarrow \text{DECAPS}(\overline{sk}, \overline{c})$ if $k = \perp \vee \overline{k} = \perp$: return 0 return $k = \overline{k} \wedge pk \neq \overline{pk} \wedge c = \overline{c}$	$g \leftarrow \mathcal{A}()$ if $g = 1$: // Scenario (I): DECAPS-DECAPS $(sk, \overline{sk}, c, \overline{c}) \leftarrow \mathcal{A}()$ $k \leftarrow \text{DECAPS}(sk, c)$ $\overline{k} \leftarrow \text{DECAPS}(\overline{sk}, \overline{c})$ if $g = 2$: // Scenario (II): ENCAPS-DECAPS $(pk, \overline{sk}, r, \overline{c}) \leftarrow \mathcal{A}()$ $(k, c) \leftarrow \text{ENCAPS}(pk; r)$ $\overline{k} \leftarrow \text{DECAPS}(\overline{sk}, \overline{c})$ if $g \notin \{1, 2\}$: // Scenario (III): ENCAPS-ENCAPS $(pk, \overline{pk}, r, \overline{r}) \leftarrow \mathcal{A}()$ $(k, c) \leftarrow \text{ENCAPS}(pk; r)$ $(\overline{k}, \overline{c}) \leftarrow \text{ENCAPS}(\overline{pk}; \overline{r})$ if $k = \perp \vee \overline{k} = \perp$: return 0 return $k = \overline{k} \wedge pk \neq \overline{pk} \wedge c = \overline{c}$
Game LEAK-BIND-K,CT-PK	
$(pk, sk) \leftarrow \text{KEYGEN}()$ $(\overline{pk}, \overline{sk}) \leftarrow \text{KEYGEN}()$ $(c, \overline{c}) \leftarrow \mathcal{A}(pk, sk, \overline{pk}, \overline{sk})$ $k \leftarrow \text{DECAPS}(sk, c)$ $\overline{k} \leftarrow \text{DECAPS}(\overline{sk}, \overline{c})$ if $k = \perp \vee \overline{k} = \perp$: return 0 return $k = \overline{k} \wedge pk \neq \overline{pk} \wedge c = \overline{c}$	

Figure 8: Security games HON-BIND-K-PK, HON-BIND-K,CT-PK, LEAK-BIND-K-PK, LEAK-BIND-K,CT-PK, MAL-BIND-K-PK, and MAL-BIND-K,CT-PK. The variants involving the ciphertext include the highlighted statements (in which case we slightly abuse the syntax by letting $\mathcal{A}$ only output a single ciphertext c), the others do not.

Game HON-BIND-K,PK-CT	Game MAL-BIND-K,PK-CT
$(pk, sk) \leftarrow \text{KEYGEN}()$ $(\overline{pk}, \overline{sk}) \leftarrow \text{KEYGEN}()$ $(pk, \overline{sk}) \leftarrow (pk, sk)$ $(c, \overline{c}) \leftarrow \mathcal{A}^{\text{Dec}, \overline{\text{Dec}}}(pk, \overline{pk})$ $k \leftarrow \text{DECAPS}(sk, c)$ $\overline{k} \leftarrow \text{DECAPS}(\overline{sk}, \overline{c})$ if $k = \perp \vee \overline{k} = \perp$: return 0 return $k = \overline{k} \wedge c \neq \overline{c}$	$g \leftarrow \mathcal{A}()$ if $g = 1$: // Scenario (I): DECAPS-DECAPS $(sk, \overline{sk}, c, \overline{c}) \leftarrow \mathcal{A}()$ $k \leftarrow \text{DECAPS}(sk, c)$ $\overline{k} \leftarrow \text{DECAPS}(\overline{sk}, \overline{c})$ if $g = 2$: // Scenario (II): ENCAPS-DECAPS $(pk, \overline{sk}, r, \overline{c}) \leftarrow \mathcal{A}()$ $(k, c) \leftarrow \text{ENCAPS}(pk; r)$ $\overline{k} \leftarrow \text{DECAPS}(\overline{sk}, \overline{c})$ if $g \notin \{1, 2\}$: // Scenario (III): ENCAPS-ENCAPS $(pk, \overline{pk}, r, \overline{r}) \leftarrow \mathcal{A}()$ $(k, c) \leftarrow \text{ENCAPS}(pk; r)$ $(\overline{k}, \overline{c}) \leftarrow \text{ENCAPS}(\overline{pk}; \overline{r})$ if $k = \perp \vee \overline{k} = \perp$: return 0 return $k = \overline{k} \wedge c \neq \overline{c} \wedge pk = \overline{pk}$
Game LEAK-BIND-K,PK-CT	
$(pk, sk) \leftarrow \text{KEYGEN}()$ $(\overline{pk}, \overline{sk}) \leftarrow \text{KEYGEN}()$ $(pk, \overline{sk}) \leftarrow (pk, sk)$ $(c, \overline{c}) \leftarrow \mathcal{A}(pk, sk, \overline{pk}, \overline{sk})$ $k \leftarrow \text{DECAPS}(sk, c)$ $\overline{k} \leftarrow \text{DECAPS}(\overline{sk}, \overline{c})$ if $k = \perp \vee \overline{k} = \perp$: return 0 return $k = \overline{k} \wedge c \neq \overline{c}$	

Figure 9: Security games X-BIND-K-CT and X-BIND-K,PK-CT for $X \in \{\text{HON}, \text{LEAK}, \text{MAL}\}$. The variants involving the public key include the highlighted statements, the other do not. In case of HON-BIND-K,PK-CT and LEAK-BIND-K,PK-CT, we will typically provide the adversary with only one public key or key pair, respectively, for sake of simplicity.

B Analysis of $\text{FO}_m^\perp$

B.1 LEAK-BIND-K,CT-PK Security for $\text{FO}_m^\perp$

In the following section, we show that $\text{FO}_m^\perp$ achieves LEAK-BIND-K,CT-PK security if and only if the underlying derandomized PKE scheme achieves our new notion SCFR-LEAK. In particular, this completes a claim made in [CDM24]: In [CDM24, Appendix B.4] LEAK-BIND-K,CT-PK security of $\text{FO}_m^\perp$ is reduced to LEAK-BIND-K,CT-PK security of the explicitly-rejecting variant $\text{FO}_m^\perp$. The latter is handled in [CDM24, Appendix B.2], which traces LEAK-BIND-K,CT-PK security back to LEAK-BIND-K,CT-PK security of $\text{FO}_m^\perp$. However, so far it is only conjectured that $\text{FO}_m^\perp$ fulfills LEAK-BIND-K,CT-PK if the underlying PKE fulfills some robustness property. Thus, the LEAK-BIND-K,CT-PK proof for $\text{FO}_m^\perp$ is not complete. We will fill this gap by providing **Theorem 16**—the proof shows that any output with at least one invalid ciphertext results in a hash collision, while the case of two valid ciphertexts contradicts the assumption on the underlying (derandomized) PKE scheme. Further, we make the robustness assumption on the PKE more precise (SCFR-LEAK) and show that it is not only a sufficient, but a necessary condition for achieving LEAK-BIND-K,CT-PK security (**Theorem 17**).

Theorem 16. *Let PKE be a public-key encryption scheme, $\text{PKE}^\mathcal{S}$ its derandomized variant, and $\text{KEM}_m^\perp$ the key-encapsulation mechanism obtained from applying $\text{FO}_m^\perp$. If $\text{PKE}^\mathcal{S}$ is*

SCFR-LEAK (resp. SCFR-CCA) secure, then KEM_m^χ is LEAK-BIND-K,CT-PK (resp. HON-BIND-K,CT-PK) secure.

Proof. Assume for a contradiction that there exists a successful adversary $\mathcal{A}$ against LEAK-BIND-K,CT-PK: given two honestly generated key pairs

$$(pk, sk^\chi = (sk, \sigma)), (\overline{pk}, \overline{sk}^\chi = (\overline{sk}, \overline{\sigma})) \leftarrow \text{KEM}_m^\chi.\text{KEYGEN}(),$$

$\mathcal{A}$ outputs a ciphertext c such that

$$k = \text{KEM}_m^\chi.\text{DECAPS}(sk^\chi, c) = \text{KEM}_m^\chi.\text{DECAPS}(\overline{sk}^\chi, c) = \overline{k}.$$

For $m = \text{PKE}^\S.\text{DEC}(sk, c)$ and $\overline{m} = \text{PKE}^\S.\text{DEC}(\overline{sk}, c)$, we distinguish the following cases:

Case 1: $m = \perp \wedge \overline{m} = \perp$ (both ciphertexts are invalid)

In this case, we have $H_U(\sigma, c) = k = \overline{k} = H_U(\overline{\sigma}, c)$. Since the keys are honestly generated, we have $\sigma \neq \overline{\sigma}$ with overwhelming probability which entails that $\mathcal{A}$ has found a collision for H_U .

Case 2: $m \neq \perp \wedge \overline{m} = \perp$ (one ciphertext is invalid)¹⁸

In this case, we have $H_U(m) = k = \overline{k} = H_U(\overline{\sigma}, c)$. Clearly, this yields a collision as $m \neq (\overline{\sigma}, c)$.

Case 3: $m \neq \perp \wedge \overline{m} \neq \perp$ (both ciphertexts are valid)

In this case, we have $H_U(m) = k = \overline{k} = H_U(\overline{m})$. Assuming that $\mathcal{A}$ does not find a collision, we can deduce $m = \overline{m}$, however, this yields that the ciphertext c also allows to win SCFR-LEAK against $\text{PKE}^\S$ as it validly decrypts to the same message under two different secret keys.

This shows LEAK-BIND-K,CT-PK security.

The very same argument applies for the notion HON-BIND-K,CT-PK, but note that the assumption SCFR-LEAK can be relaxed to SCFR-CCA. The reason is that game HON-BIND-K,CT-PK no longer grants the secret key but a decryption oracle to the adversary. Hence the reduction merely needs access to a decryption oracle to simulate the view of the adversary. This concludes the proof. $\square$

Remark 4. The above theorem extends to the new LEAK^{+r} model, introduced in [FG25], when the underlying PKE scheme satisfies the natural extension of SCFR-LEAK to SCFR-LEAK^{+r} , which provides the adversary with the randomness used to generate the keys.¹⁹ Note that every other binding result from this paper directly extends to LEAK^{+r} , as we either have security in the stronger MAL setting or attacks in the weaker LEAK setting.

The following theorem shows that SCFR-LEAK is not only a sufficient but also a necessary condition for achieving LEAK-BIND-K,CT-PK security.

Theorem 17. *Let PKE be a PKE scheme, $\text{PKE}^\S$ its derandomized variant, and KEM_m^χ the KEM obtained from applying FO_m^χ . If KEM_m^χ is LEAK-BIND-K,CT-PK secure, then $\text{PKE}^\S$ is SCFR-LEAK secure.*

Proof. Assume for a contradiction that $\text{PKE}^\S$ is not SCFR-LEAK secure, i.e., there is an adversary $\mathcal{A}$ that obtains honestly generated key pairs (pk, sk) , $(\overline{pk}, \overline{sk})$ and outputs c s.t. $\text{PKE}^\S.\text{DEC}(sk, c) = m = \overline{m} = \text{PKE}^\S.\text{DEC}(\overline{sk}, c) \neq \perp$. As $m = \overline{m} \neq \perp$, we obtain

¹⁸Here we assume wlog that $\overline{c}$ is invalid.

¹⁹Note that the separation example given in [FG25] can be extended to separate SCFR-LEAK and SCFR-LEAK^{+r} .

that $k = H_U(m) = H_U(\bar{m}) = \bar{k}$. Then we can construct an adversary $\mathcal{B}$ against KEM_m^χ that simply outputs the ciphertext c that $\mathcal{A}$ outputs (after providing $\mathcal{A}$ with the same key-pair it got from the LEAK-BIND-K,CT-PK game minus the implicit rejection values σ and $\bar{\sigma}$). Adversary $\mathcal{B}$ is clearly also successful in winning LEAK-BIND-K,CT-PK which contradicts the assumption that KEM_m^χ is LEAK-BIND-K,CT-PK secure and thus finishes the proof. $\square$

The corollary below shows that—unlike for FO_m^χ —the notions X-BIND-K-PK and X-BIND-K,CT-PK are not equivalent (for $X = \text{HON}$ and $X = \text{LEAK}$). This follows—for KEMs constructed via the FO_m^χ transform—from our positive result for X-BIND-K,CT-PK (Theorem 16) and the negative result for X-BIND-K,CT-PK (Theorem 4).

Corollary 1. *There is a PKE scheme PKE, such that the KEM KEM_m^χ resulting from applying FO_m^χ to PKE is X-BIND-K,CT-PK secure but not X-BIND-K-PK secure, for $X \in \{\text{HON}, \text{LEAK}\}$.*

B.2 MAL-BIND-K,PK-CT Security for FO_m^χ

We show that FO_m^χ achieves X-BIND-K,PK-CT for $X \in \{\text{HON}, \text{LEAK}, \text{MAL}\}$ which, together with Theorem 2, shows that these notions are achieved for any implicitly-rejecting FO-KEM.

The theorem below shows that key-encapsulation mechanisms obtained via the FO_m^χ transform achieve MAL-BIND-K,PK-CT security. By hierarchy, this implies security wrt LEAK-BIND-K,PK-CT and HON-BIND-K,PK-CT. The proof shows that for all possible attacks, the adversary needs to find a collision for the hash function.

Theorem 18. *Let PKE be a PKE scheme and KEM_m^χ the KEM obtained from applying FO_m^χ to PKE. Then KEM_m^χ is X-BIND-K,PK-CT secure for $X \in \{\text{MAL}, \text{LEAK}, \text{HON}\}$.*

Proof. Note that for MAL-BIND-K,PK-CT, there are three different variants to consider: (I) DECAPS-DECAPS, (II) ENCAPS-DECAPS, and (III) ENCAPS-ENCAPS. We give the proof for the first variant and subsequently argue why it also covers the other two variants.

Adversary $\mathcal{A}$ needs to output two secret keys $sk^\chi = (sk, \sigma)$ and $\bar{sk}^\chi = (\bar{sk}, \bar{\sigma})$ (it must hold that the implicitly contained public keys are not distinct, i.e., $pk = \bar{pk}$) and two distinct ciphertexts $c \neq \bar{c}$ that result in the same shared key k when decrypting under the respective secret keys. We can distinguish between the following cases for the ciphertexts output by $\mathcal{A}$, where $m \leftarrow \text{PKE}^\S.\text{DEC}(sk, c)$ and $\bar{m} \leftarrow \text{PKE}^\S.\text{DEC}(\bar{sk}, \bar{c})$:

Case 1: $m = \perp \wedge \bar{m} = \perp$ (both ciphertexts are invalid)

In this case, we have $H_U(\sigma, c) = k = \bar{k} = H_U(\bar{\sigma}, \bar{c})$. Even though $\mathcal{A}$ has full control over σ and $\bar{\sigma}$, the fact that $c \neq \bar{c}$ holds, implies that $\mathcal{A}$ has to find a collision for H_U ;

Case 2: $m \neq \perp \wedge \bar{m} = \perp$ (one ciphertext is invalid)²⁰

In this case, we obtain $H_U(m) = k = \bar{k} = H_U(\bar{\sigma}, \bar{c})$. Clearly, this would also entail a collision for H_U ;

Case 3: $m \neq \perp \wedge \bar{m} \neq \perp$ (both ciphertexts are valid)

In this case, it holds that $H_U(m) = k = \bar{k} = H_U(\bar{m})$. We can deduce $m = \bar{m}$ as otherwise, we would again have a collision for $H_{T_\S}$. However, in this case, validity of both ciphertexts yields

$$c = \text{ENC}(pk, m; H_{T_\S}(m)) = \text{ENC}(\bar{pk}, \bar{m}; H_{T_\S}(\bar{m})) = \bar{c},$$

where the second equality follows since both $pk = \bar{pk}$ and $m = \bar{m}$, hence yielding a contradicting to $c \neq \bar{c}$.

²⁰Here we assume wlog that $\bar{c}$ is invalid.

In the following, we argue that the above three cases also cover the other variants, i.e., (II) ENCAPS-DECAPS, and (III) ENCAPS-ENCAPS. In a nutshell, if $\mathcal{A}$ provides the randomness for ENCAPS instead of a ciphertext (and a public key instead of a secret key), correctness of the KEM yields that the resulting ciphertext will be valid, meaning that some of the above cases simply cannot occur.

For the ENCAPS-DECAPS variant, $\mathcal{A}$ will output a message m from which the ciphertext $c \leftarrow \text{KEM}_m^\perp.\text{ENCAPS}(pk; m)$ is derived. By correctness of $\text{KEM}_m^\perp$, this yields that c will be a valid ciphertext, hence excluding the first case above. For the other two scenarios, the above argumentation can be applied.

For the ENCAPS-ENCAPS variant, $\mathcal{A}$ will output two messages m and $\bar{m}$ from which the ciphertext will be derived as $c \leftarrow \text{KEM}_m^\perp.\text{ENCAPS}(pk; m)$ and $\bar{c} \leftarrow \text{KEM}_{\bar{m}}^\perp.\text{ENCAPS}(\bar{pk}; \bar{m})$, respectively. We thus get that both ciphertexts will be valid by correctness, which means we merely need the final case from the three above.

By the hierarchy of the notions, the above also shows LEAK-BIND-K,PK-CT security and HON-BIND-K,PK-CT security. This concludes the proof. $\square$

C Postponed Proofs

C.1 Proof of Theorem 9

Proof. Insecurity with respect to LEAK-BIND-K,CT-PK follows easily from Theorem 5 using the fact that HQC.PKE fulfills restricted non-rigidity. The latter is proven in Proposition 3, which covers BIKE, HQC, and the modified variant HQC*. While Theorem 5 is formulated for $\text{FO}^\perp$, it directly translates to the variant used by HQC. Then HQC is also insecure wrt the notions LEAK-BIND-K-PK, MAL-BIND-K,CT-PK, and MAL-BIND-K-PK by the established hierarchy. $\square$

C.2 Proof of Theorem 10

Proof. We construct the following LEAK-BIND-K,PK-CT adversary $\mathcal{A}$ against HQC. Its input is a key-pair $(pk, sk^\perp = (sk, \sigma))$, where $pk = (\mathbf{h}, \mathbf{s})$ and $sk = (\mathbf{x}, \mathbf{y})$, and it is supposed to output distinct ciphertexts c and $\bar{c}$ such that

$$\text{HQC.DECAPS}(sk^\perp, c) = k = \bar{k} = \text{HQC.DECAPS}(sk^\perp, \bar{c}).$$

Adversary $\mathcal{A}$ first picks $\text{salt} \leftarrow \mathcal{S}$, sets $m \leftarrow \sigma$, and computes the ciphertext $c = (\mathbf{u}, \mathbf{v}, \text{salt}) \leftarrow \text{HQC.ENCAPS}(pk; m, \text{salt})$. By construction, we have $\mathbf{u} = \mathbf{r}_1 + \mathbf{h}\mathbf{r}_2$ for $(\mathbf{e}, \mathbf{r}_1, \mathbf{r}_2) = \text{H}_{\text{T}_g}(m, pk, \text{salt})$. Next, $\mathcal{A}$ picks $\bar{\text{salt}} \leftarrow \mathcal{S} \setminus \{\text{salt}\}$, and outputs the two ciphertexts $c \leftarrow (\mathbf{u}, \mathbf{v}, \text{salt})$ and $\bar{c} \leftarrow (\mathbf{u}, \mathbf{v}, \bar{\text{salt}})$.

The two ciphertexts are distinct as we have $\text{salt} \neq \bar{\text{salt}}$. It remains to argue that they result in the same key. By correctness, with overwhelming probability, we have

$$k = \text{HQC.DECAPS}(sk^\perp, (\mathbf{u}, \mathbf{v}, \text{salt})) = \text{H}_U(m, (\mathbf{u}, \mathbf{v})).$$

Next, we consider the output of the decapsulation algorithm for the second ciphertext $(\mathbf{u}, \mathbf{v}, \bar{\text{salt}})$, i.e., $\text{HQC.DECAPS}(sk^\perp, (\mathbf{u}, \mathbf{v}, \bar{\text{salt}}))$. We argue that the recomputation of the ciphertext-part $(\mathbf{u}, \mathbf{v})$ will fail with overwhelming probability. By correctness, we get $m = \text{DECODER}(\mathbf{v} - \mathbf{u}\mathbf{y})$ as the ciphertext was honestly generated and is decrypted (using the underlying base PKE HQC.PKE) using the same secret key. For the recomputation, the randomness $(\bar{\mathbf{e}}, \bar{\mathbf{r}}_1, \bar{\mathbf{r}}_2) = \text{H}_{\text{T}_g}(m, pk, \bar{\text{salt}})$ is used which will be different from $(\mathbf{e}, \mathbf{r}_1, \mathbf{r}_2) = \text{H}_{\text{T}_g}(m, pk, \text{salt})$ as otherwise, $\mathcal{A}$ would have found a collision for H_{T_g} . In the following, we assume that $(\bar{\mathbf{r}}_1, \bar{\mathbf{r}}_2) \neq (\mathbf{r}_1, \mathbf{r}_2)$, which can be easily achieved by letting $\mathcal{A}$ sample $\bar{\text{salt}}$

until that is the case. Let $(\bar{\mathbf{u}}, \bar{\mathbf{v}})$ denote the recomputed ciphertext (using randomness $(\bar{\mathbf{e}}, \bar{\mathbf{r}}_1, \bar{\mathbf{r}}_2)$). This leads to

$$(\mathbf{u}, \mathbf{v}) = ((\mathbf{r}_1 + \mathbf{h}\mathbf{r}_2), \mathbf{v}) \neq ((\bar{\mathbf{r}}_1 + \mathbf{h}\bar{\mathbf{r}}_2), \bar{\mathbf{v}}) = (\bar{\mathbf{u}}, \bar{\mathbf{v}}).$$

Thus the ciphertext gets rejected by outputting $\bar{k} \leftarrow \mathbf{H}_U(\sigma, (\mathbf{u}, \mathbf{v}))$. Using the choice of m , we get $\bar{k} = \mathbf{H}_U(\sigma, (\mathbf{u}, \mathbf{v})) = \mathbf{H}_U(m, (\mathbf{u}, \mathbf{v})) = k$ which shows that adversary $\mathcal{A}$ wins the game LEAK-BIND-K,PK-CT. $\square$

C.3 Proof of Theorem 11

Proof. We give the proof for HON-BIND-K-PK which extends to HON-BIND-K,CT-PK by the hierarchy of the notions.

Assume, for sake of contradiction, that there is an adversary $\mathcal{A}$ that wins the game HON-BIND-K-PK, i.e., given two honestly generated key pairs $(pk, sk^\perp)$ and $(\bar{pk}, \bar{sk}^\perp)$, with $sk^\perp = (sk, \sigma)$ and $\bar{sk}^\perp = (\bar{sk}, \bar{\sigma})$, it outputs $c, \bar{c}$ such that $\text{HQC.DECAPS}(sk^\perp, c) = k = \bar{k} = \text{HQC.DECAPS}(\bar{sk}^\perp, \bar{c})$. We distinguish between the cases $c \neq \bar{c}$ and $c = \bar{c}$ and start with the former.

Firstly note that $c \neq \bar{c}$ can be further divided in the following cases:

$$\begin{aligned} (\mathbf{u}, \mathbf{v}) &\neq (\bar{\mathbf{u}}, \bar{\mathbf{v}}) \wedge \text{salt} \neq \bar{\text{salt}} \\ (\mathbf{u}, \mathbf{v}) &\neq (\bar{\mathbf{u}}, \bar{\mathbf{v}}) \wedge \text{salt} = \bar{\text{salt}} \\ (\mathbf{u}, \mathbf{v}) &= (\bar{\mathbf{u}}, \bar{\mathbf{v}}) \wedge \text{salt} \neq \bar{\text{salt}} \end{aligned}$$

Independent of the fact whether the ciphertexts are rejected or not, the computation of the keys includes $(\mathbf{u}, \mathbf{v})$ and $(\bar{\mathbf{u}}, \bar{\mathbf{v}})$, more precisely $k = \mathbf{H}_U(\cdot, (\mathbf{u}, \mathbf{v}))$ and $\bar{k} = \mathbf{H}_U(\cdot, (\bar{\mathbf{u}}, \bar{\mathbf{v}}))$. As $k = \bar{k}$, this implies that the first two cases mentioned above cannot occur, as otherwise the adversary would have found a collision for $\mathbf{H}_U$.²¹ This leaves us with the case that $(\mathbf{u}, \mathbf{v}) = (\bar{\mathbf{u}}, \bar{\mathbf{v}})$ and $\text{salt} \neq \bar{\text{salt}}$. Then, we can distinguish between the following cases for the ciphertexts output by $\mathcal{A}$ and $m \leftarrow \text{PKE}^\S.\text{DEC}(sk, c)$, $\bar{m} \leftarrow \text{PKE}^\S.\text{DEC}(\bar{sk}, \bar{c})$:

Case 1: $m = \perp \wedge \bar{m} = \perp$ (both ciphertexts are invalid)

Then $k = \mathbf{H}_U(\sigma, (\mathbf{u}, \mathbf{v})) = \mathbf{H}_U(\bar{\sigma}, (\bar{\mathbf{u}}, \bar{\mathbf{v}})) = \bar{k}$, which implies $\sigma = \bar{\sigma}$ as otherwise $\mathcal{A}$ would have found a collision for $\mathbf{H}_U$. However, the keys are honestly generated, thus the randomly chosen rejection values will differ with overwhelming probability.

Case 2: $m \neq \perp \wedge \bar{m} = \perp$ (one ciphertext is invalid)²²

Then $k = \mathbf{H}_U(m, (\mathbf{u}, \mathbf{v})) = \mathbf{H}_U(\bar{\sigma}, (\bar{\mathbf{u}}, \bar{\mathbf{v}})) = \bar{k}$, which implies $m = \bar{\sigma}$ as otherwise $\mathcal{A}$ would have found a collision for $\mathbf{H}_U$. However, $m = \bar{\sigma}$ can be excluded with overwhelming probability, as $\mathcal{A}$ does not get $\bar{\sigma}$ and its only access is via querying invalid ciphertexts to the decryption oracle in which case the response will be the output of a random oracle on $\bar{\sigma}$ and the queried ciphertext which also does not reveal $\bar{\sigma}$.

Case 3: $m \neq \perp \wedge \bar{m} \neq \perp$ (both ciphertexts are valid)

Then $k = \mathbf{H}_U(m, (\mathbf{u}, \mathbf{v})) = \mathbf{H}_U(\bar{m}, (\bar{\mathbf{u}}, \bar{\mathbf{v}})) = \bar{k}$, which implies $m = \bar{m}$ as otherwise $\mathcal{A}$ would have found a collision for $\mathbf{H}_U$. In total $\mathcal{A}$ found $c, \bar{c}$ such that $\text{PKE}^\S.\text{DEC}(sk, (\mathbf{u}, \mathbf{v})) = \text{PKE}^\S.\text{DEC}(\bar{sk}, (\bar{\mathbf{u}}, \bar{\mathbf{v}})) \neq \perp$, however, this contradicts the fact that HQC is SCFR-CCA secure, which is proven in Proposition 3.

Lastly, we consider the case that $c = \bar{c}$, i.e., $(\mathbf{u}, \mathbf{v}) = (\bar{\mathbf{u}}, \bar{\mathbf{v}})$ and $\text{salt} = \bar{\text{salt}}$. For this we can apply the same argument as was used above for the case that $(\mathbf{u}, \mathbf{v}) = (\bar{\mathbf{u}}, \bar{\mathbf{v}})$ and $\text{salt} \neq \bar{\text{salt}}$ holds. This part of the proof relies only on the fact that $(\mathbf{u}, \mathbf{v}) = (\bar{\mathbf{u}}, \bar{\mathbf{v}})$ —at no point it is used that $\text{salt} \neq \bar{\text{salt}}$. This finishes the proof for HON-BIND-K-PK. $\square$

²¹Note that this argument is essentially the proof of Theorem 2.

²²Here we assume wlog that $\bar{c}$ is invalid.

C.4 Proof of Theorem 12

Proof. We give an attack against HON-BIND-K,PK-CT which, by the hierarchy of the notions, extends to HON-BIND-K-CT. We construct the following adversary $\mathcal{A}$. It receives a single public key pk as input. Adversary $\mathcal{A}$ picks an arbitrary ciphertext $c = (\mathbf{u}, \mathbf{v}, salt)$ and sets $\bar{c} = (\mathbf{u}, \mathbf{v}, \bar{salt})$ with $\bar{salt} \neq salt$. With overwhelming probability these two ciphertexts will be invalid. In this case, we obtain the same shared key

$$k = \text{HQC.DECAPS}(sk, c) = \text{H}_U(\sigma, (\mathbf{u}, \mathbf{v})) = \text{HQC.DECAPS}(sk, \bar{c}) = \bar{k}.$$

Thus, $\mathcal{A}$ wins the game HON-BIND-K,PK-CT. $\square$

C.5 Proof of Proposition 2

Proof. We will give the proof for T_g and T_s simultaneously and mark the changes from T_g to T_s using gray boxes.

1. Let $\mathcal{A}$ be an SCFR-LEAK adversary against $\mathsf{X}[\text{PKE}, \mathsf{H}_T]$. That means given (pk, sk) and $(pk, \bar{sk})$, both stemming from honest key generations, $\mathcal{A}$ outputs a ciphertext c .²³ To show SCFR-LEAK security, we need to check that

$$m = \mathsf{X}[\text{PKE}, \mathsf{H}_T].\text{DEC}(sk, c) = \mathsf{X}[\text{PKE}, \mathsf{H}_T].\text{DEC}(\bar{sk}, c) \neq \perp.$$

happens only with negligible probability. For X the T_g transform (and the T_s transform, respectively), due to the re-encryption check, this translates to

$$\text{PKE.ENC}(pk, m; \mathsf{H}_T(m, pk, salt)) = c = \text{PKE.ENC}(\bar{pk}, m; \mathsf{H}_T(m, \bar{pk}, salt)).$$

The probability for this to happen is negligible by the assumption that two encryptions using honestly generated public keys $pk, \bar{pk}$ and random values $\mathsf{H}_T(m, pk, salt)$, $\mathsf{H}_T(m, \bar{pk}, salt)$ will result in different ciphertexts with overwhelming probability.

2. We consider two honestly generated key pairs $(pk, sk), (\bar{pk}, \bar{sk})$, a randomly chosen message m , and $c \leftarrow \text{PKE.ENC}(pk, m)$. To check restricted non-rigidity for PKE, we need to verify that

$$\text{PKE.ENC}(\bar{pk}, \text{PKE.DEC}(\bar{sk}, c)) \neq c,$$

holds with overwhelming probability. This follows directly from our assumption that encryption with the two honestly generated public keys $pk, \bar{pk}$ using random coins will result in different ciphertexts with overwhelming probability. $\square$

C.6 Proof of Proposition 3

Proof. Using Proposition 2 it suffices to show that ciphertexts of the PKE schemes $\mathsf{T}_g[\text{BIKE.PKE}, \mathsf{H}_T]$ and $\mathsf{T}_s[\text{HQC.PKE}, \mathsf{H}_T]$ have the following property: For two honestly generated public keys $pk, \bar{pk}$ and two (not necessarily different) messages $m, \bar{m}$, the ciphertexts $c = \text{ENC}(pk, m)$ and $\bar{c} = \text{ENC}(\bar{pk}, \bar{m})$ differ.

In the following, we focus on the first ciphertext component of the PKE schemes $\mathsf{T}_g[\text{BIKE.PKE}, \mathsf{H}_T]$ and $\mathsf{T}_s[\text{HQC.PKE}, \mathsf{H}_T]$ to show that we obtain different ciphertexts in the above situation²⁴: (1) For BIKE, consider $r_B = e_0 + e_1 h$, where $(e_0, e_1) = \mathsf{H}_{T_g}(m)$ and

²³Remember that for T_s , the ciphertext contains a salt.

²⁴Note that for HQC, we could also argue over the salt value included in each ciphertext, which is chosen randomly during encapsulation and hence differs for different ciphertexts.

$h = pk$. (2) For HQC, consider $r_H = \mathbf{r}_1 + \mathbf{h}\mathbf{r}_2$, where $(\mathbf{r}_1, \mathbf{r}_2) = \mathbf{H}_{\mathbb{T}_s}(m, pk, salt)$ and h is part of the public key. To check the property, we consider honestly generated public keys $pk_B, \overline{pk}_B$ for BIKE and $pk_H, \overline{pk}_H$ for HQC as well as two messages $m, \overline{m}$. Denote the first ciphertext components resulting from encrypting m with pk_B (and pk_H) by r_B (and r_H), and the ones resulting from encrypting $\overline{m}$ with $\overline{pk}_B$ (and $\overline{pk}_H$) by $\overline{r}_B$ (and $\overline{r}_H$). Then, we obtain

$$\begin{aligned} r_B &= e_0 + e_1 h \neq \overline{e}_0 + \overline{e}_1 \overline{h} = \overline{r}_B \\ r_H &= \mathbf{r}_1 + \mathbf{h}\mathbf{r}_2 \neq \overline{\mathbf{r}}_1 + \overline{\mathbf{h}}\overline{\mathbf{r}}_2 = \overline{r}_H \end{aligned}$$

with overwhelming probability as $h, \overline{h}, \mathbf{h}$, and $\overline{\mathbf{h}}$ are randomly chosen values for honestly generated keys, and (e_0, e_1) and $(\mathbf{r}_1, \mathbf{r}_2)$ are derived using random oracles. This finishes the proof. $\square$

C.7 Proof of Theorem 14

Proof. We cover the proof for both notions simultaneously, emphasizing the relevant differences when necessary. In both games, adversary $\mathcal{A}$ receives two honestly generated secret keys $sk^\mathcal{L} = (sk, \sigma), \overline{sk}^\mathcal{L} = (\overline{sk}, \overline{\sigma})$ —note that we assume the corresponding public keys $(pk/\overline{pk})$ and their hash values $(h_{pk}/h_{\overline{pk}})$ to be included in the secret keys—and has to output two ciphertexts $c, \overline{c}$ that decapsulate to the same shared key.

Depending on the validity of the ciphertexts (wrt PKE), we can distinguish between three cases. For each of them, we show that for $\mathcal{A}$ to be successful, there would have to be a random oracle collision, which can only happen with negligible probability.

Case 1: $m = \perp \wedge \overline{m} = \perp$ (both ciphertexts are invalid)

In this case, we have $\mathbf{H}_U(\sigma, c) = k = \overline{k} = \mathbf{H}_U(\overline{\sigma}, \overline{c})$. Since the keys are honestly generated, we have $\sigma \neq \overline{\sigma}$ with overwhelming probability which entails that $\mathcal{A}$ has found a collision for $\mathbf{H}_U$. (In case of LEAK-BIND-K-CT we get a collision for $\mathbf{H}_U$ with certainty as the ciphertexts output by $\mathcal{A}$ have to be distinct.)

Case 2: $m \neq \perp \wedge \overline{m} = \perp$ (one ciphertext is invalid)

In this case, we have $\mathbf{H}_U(m, h_{pk}) = k = \overline{k} = \mathbf{H}_U(\overline{\sigma}, \overline{c})$. This yields a collision as $(m, h_{pk}) \neq (\overline{\sigma}, \overline{c})$ due to the second inputs differing (one is a hash value, the other a ciphertext).

Case 3: $m \neq \perp \wedge \overline{m} \neq \perp$ (both ciphertexts are valid)

In this case, we have $\mathbf{H}_U(m, h_{pk}) = k = \overline{k} = \mathbf{H}_U(\overline{m}, \overline{h}_{pk})$. Since the keys are honestly generated, we have $pk \neq \overline{pk}$ with overwhelming probability and thus a collision for $\mathbf{H}_U$ or $\mathbf{H}$.

This concludes the proof. $\square$

C.8 Proof of Theorem 15

Proof. We give the proof for MAL-BIND-K-PK and MAL-BIND-K-CT for scenario (I) DECAPS-DECAPS. At the end of the proof, we argue why it also covers the other two cases: (II) ENCAPS-DECAPS and (III) ENCAPS-ENCAPS.

Adversary $\mathcal{A}$ outputs two secret keys $sk^\mathcal{L} = (sk, \sigma), \overline{sk}^\mathcal{L} = (\overline{sk}, \overline{\sigma})$ (which contain the corresponding public keys $pk, \overline{pk}$) and two ciphertexts $c, \overline{c}$ that decapsulate to the same shared keys. Depending on the validity of the ciphertexts (wrt PKE), we can distinguish between three cases. For all of them (except the last one), $\mathcal{A}$ can only win if it finds a random oracle collision, which we assume to be impossible.

Case 1: $m = \perp \wedge \bar{m} = \perp$ (both ciphertexts are invalid)

In this case, we have $H_U(\sigma, H(pk), c) = k = \bar{k} = H_U(\bar{\sigma}, H(\bar{pk}), \bar{c})$. Depending on the notion the public keys (MAL-BIND-K-PK) or the ciphertexts (MAL-BIND-K-CT) have to be different. In both cases, we can therefore conclude that the adversary needs to find a collision in the hash function H_U (or H in case of MAL-BIND-K-PK where also $H(pk) = H(\bar{pk})$ can hold for $pk \neq \bar{pk}$).

Case 2: $m \neq \perp \wedge \bar{m} = \perp$ (one ciphertext is invalid)

In this case, we have $H_U(m, H(pk)) = k = \bar{k} = H_U(\bar{\sigma}, H(\bar{pk}), \bar{c})$. Clearly, this yields a collision as $(m, H(pk)) \neq (\bar{\sigma}, H(\bar{pk}), \bar{c})$.

Case 3: $m \neq \perp \wedge \bar{m} \neq \perp$ (both ciphertexts are valid)

In this case, we have $H_U(m, H(pk)) = k = \bar{k} = H_U(\bar{m}, H(\bar{pk}))$ and need to further distinguish between the two notions:

Case 3.1: MAL-BIND-K-PK

Since the output by the adversary for MAL-BIND-K-PK must satisfy $pk \neq \bar{pk}$, this yields that $\mathcal{A}$ has found a collision for H_U or H .

Case 3.2: MAL-BIND-K-CT

Assuming that the adversary does not find collisions for any of the hash functions H_U , H , and H_T , equality of the shared keys k and $\bar{k}$ yields $pk = \bar{pk}$ and $m = \bar{m}$ (for the messages resulting from decrypting c and $\bar{c}$ using sk and $\bar{sk}$, respectively). Since both ciphertexts are valid, the re-encryption succeeds for both, i.e.,

$$c = \text{PKE.ENC}(pk, m; H_T(m)) = \text{PKE.ENC}(\bar{pk}, \bar{m}; H_T(\bar{m})) = \bar{c}.$$

Then, however, $\mathcal{A}$ cannot win MAL-BIND-K-CT, as the game requires different ciphertexts.

The above cases also cover the ENCAPS-DECAPS scenario and ENCAPS-ENCAPS scenario. Here, at least one ciphertext is honestly generated, meaning that it will be a valid ciphertext as encryption does not fail. This means that certain cases cannot occur. More precisely, only the last two cases can appear for ENCAPS-DECAPS while only the last case can for ENCAPS-ENCAPS.

By the hierarchy of the notions, the above also shows security wrt to the other binding notions. This concludes the proof. $\square$